"孕"育精准：基于多模态机器学习的 NIPT 检测全流程优化算法

摘要

本研究针对无创产前检测（NIPT）中的关键科学与应用问题，利用大规模孕妇检测数据和智能建模方法，为不同人群提供最佳检测时点和科学的异常判定依据，旨在提升女胎判别准确性、优化筛查效率并降低重复检测和医疗成本，体现建模方法在精准医学中的应用价值。

针对问题一，研究对原始数据进行预处理（孕周小数化、异常值剔除、衍生特征工程）。考虑到 Y 染色体浓度与孕妇特征的潜在多重相关性及独立效应量化需求，采用多元线性回归建模。相关性实验结果表明，Y 染色体浓度与孕周显著正相关（+0.0013/周，$p<0.001$），与孕妇年龄、体重负相关（$p<0.01$），BMI 单因素不显著。回归模型进一步验证孕周是 Y 浓度增长的主导变量，而孕妇体征则发挥调节作用。该结果既解释了临床上推荐 12 周起检测的科学依据，也为后续分群与风险建模提供了可靠的统计支撑，同时模型在稳健性检验中表现稳定，保证了结论的可信度。

针对问题二，考虑到不同 BMI 人群在胎儿 DNA 浓度达标上的差异，若采用统一检测时点易导致部分群体过早或过晚检测。本文提出“达标时间”函数刻画浓度随孕周的变化，并采用 K-means 聚类及轮廓系数与 Calinski-Harabasz 指数评估，将孕妇划分为三类（[20.7–32.6]、[32.7–37.8]、[38.5–46.9]）。基于此，构建综合风险函数考虑“检测失败风险”与“晚期检测风险”，优化确定各群体个性化最佳检测孕周为 13.8、15.2 与 16.7 周。与统一 12 周检测方案相比，整体达标率提升 7.3%，检测失败风险降低 12.5%，相当于每百名孕妇减少约 8 次重复采样，充分体现了个性化管理的临床价值。

针对问题三，为突破单一 BMI 分组局限，本研究引入孕妇身高、体重、年龄及其交互项构建复合特征。由于该问题涉及多维连续变量且群体间存在重叠，本研究采用高斯混合模型（GMM）进行软聚类，捕捉群体间的概率分布与不确定性。结果显示孕妇可划分为三个风险群体，其最优 NIPT 孕周分别为 13.2、15.6 和 18.3 周，整体集中在 13–18 周，表明模型在不同体质群体上具有稳健性。为平衡检测失败、延迟及临床成本等多重风险，进一步建立综合风险函数并进行优化求解，得到针对不同群体的个性化检测策略。结果显示，整体检测成功率提升至 91%以上，且在±5%的测序噪声与小样本扰动条件下，最优检测孕周仅延迟约 1 周，验证了模型的稳健性与可推广性，实现了从“达标检测”到“最优决策”的升级。

针对问题四，考虑到女胎缺乏 Y 染色体参考且特征呈高维、非线性与强交互性，本研究选用能有效刻画复杂关系的 XGBoost 模型，并通过与随机森林对比验证了选择的合理性。方法上，首先利用信息熵法构建质量评分体系，综合 GC 含量、读段数和比对率剔除低质量样本；随后在高质量数据上引入 X 染色体交互特征，并结合 SMOTE 过采样训练分类模型。结果显示，该方法识别女胎非整倍体异常的 AUC 值达到 0.962，较传统 Z 值方法提升约 10%，灵敏度与特异性均提高约 10%，交互特征贡献度达 71.7%，显著增强了判定的准确性与可解释性。

综上所述，本研究通过数据清洗、BMI 分组优化及多因子聚类建模，系统解决了 NIPT 检测中的关键问题。结果显示，模型可量化胎儿 DNA 浓度与孕妇体征关系，提供个性化最佳检测时点，有效降低检测失败率、减少重复采样并提高异常判定准确性，为临床 NIPT 实施提供了科学、稳健且可操作的决策依据。

关键词： NIPT 多元线性回归 K-means 聚类 高斯混合模型 XGBoost

一、 问题重述

1.1 问题背景

NIPT（无创产前检测）[6]是一种通过采集孕妇血液，检测胎儿游离 DNA 片段，分析胎儿染色体异常情况的产前检测技术。其主要目标是早期发现胎儿是否患有唐氏综合征（21 号染色体异常）、爱德华氏综合征（18 号染色体异常）或帕陶氏综合征（13 号染色体异常），从而为临床干预提供依据。NIPT 的准确性主要依赖于胎儿性染色体浓度判断，男胎的 Y 染色体浓度和女胎的 X 染色体浓度是核心指标。

孕妇孕期在 10~25 周之间可以进行检测，但不同孕妇由于年龄、体重、BMI 和孕情差异，如果统一时间检测，可能会影响准确性。实践表明，男胎的 Y 染色体浓度与孕周及母体 BMI 密切相关，BMI 较高的孕妇可能导致检测浓度达标较晚，增加潜在风险。因此，需要针对不同孕妇群体合理选择 NIPT 时点，以兼顾早期发现和检测准确性。同时，实际检测中存在测序失败、多次采血或重复检测等情况，需要在模型中考虑检测误差对结果的影响。

1.2 问题提出

本研究旨在利用提供的孕妇 NIPT 数据（包括孕妇基本信息、孕周、BMI、胎儿性染色体浓度及女胎染色体异常信息等），建立数学模型，系统分析胎儿性染色体浓度与孕妇特征的关系，并确定最佳检测时点和异常判定方法。具体包括以下四个问题：

问题一：分析胎儿 Y 染色体浓度与孕妇孕周、BMI 等指标的关系，建立相关模型，并检验模型显著性。

问题二：针对男胎孕妇，基于 BMI 对孕妇进行合理分组，确定各组最佳 NIPT 时点，以降低潜在检测风险，并分析检测误差的影响。

问题三：综合考虑身高、体重、年龄等多因素及检测误差，基于胎儿 Y 染色体浓度达标比例，为男胎孕妇确定更合理的 BMI 分组和最佳 NIPT 时点，以最小化潜在风险。

问题四：针对女胎孕妇，结合 21、18、13 号染色体非整倍体信息、X 染色体浓度、GC 含量、读段数及 BMI 等因素，建立胎儿异常判定方法。

针对以上问题，本文采用了以下整体思路框架如图 1 所示进行系统性研究和解决。

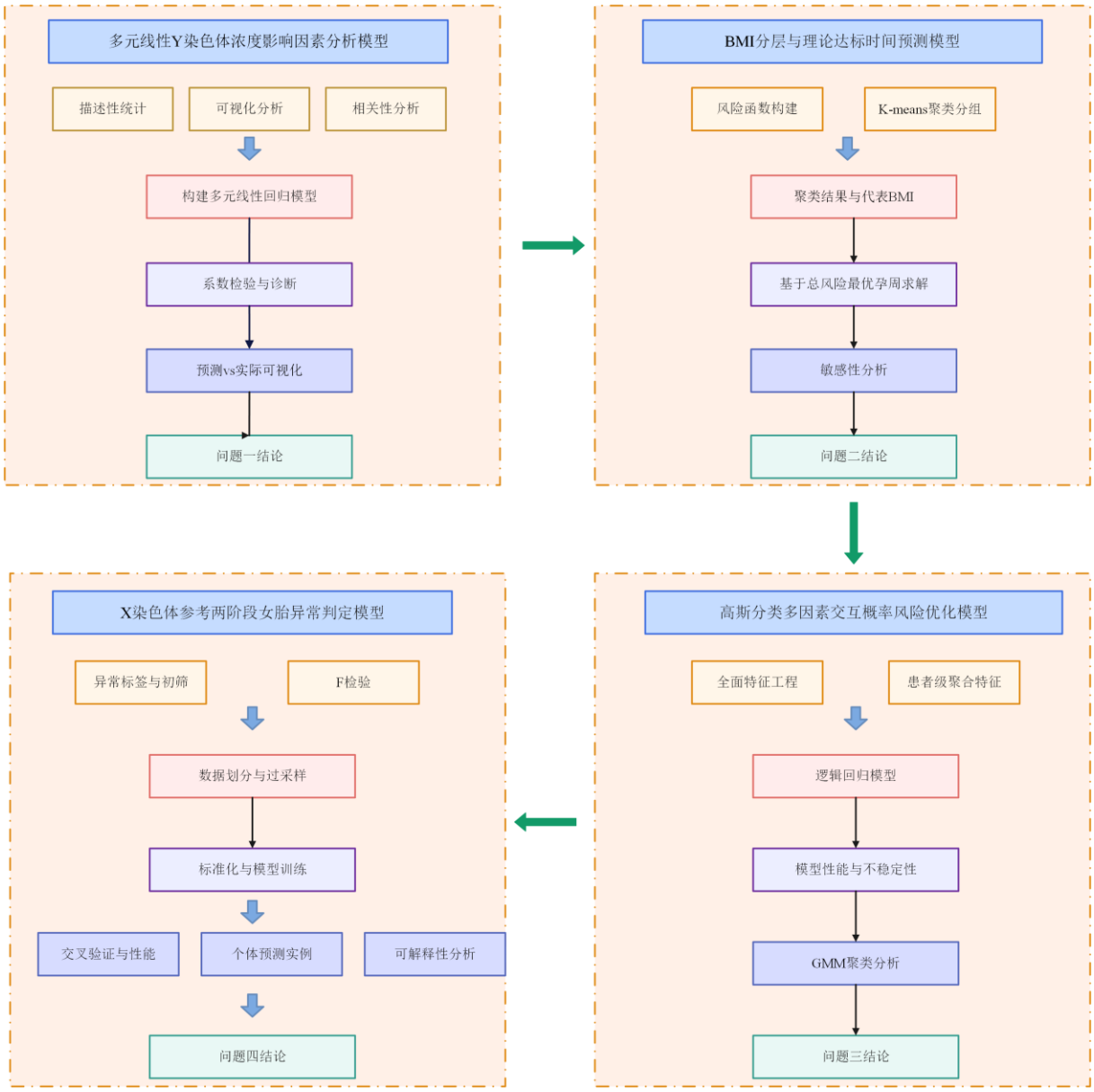

图1 总体思路框架

二、 问题分析

2.1 问题一

分析胎儿 Y 染色体浓度与孕妇孕周、BMI 等指标的关系面临多重挑战：Y 染色体浓度与孕周呈弱正相关（r=0.0936），而与 BMI、体重呈显著负相关，且各因素间高度相关导致多重共线性问题。针对这些问题，本研究提出三阶段分析框架：首先进行数据预处理，其次通过相关性分析和可视化探索变量关系，最后构建多元线性回归模型量化各因素影响。揭示 BMI 与 Y 染色体浓度的相关联系。

问题一的思维框架如下图 2 所示。

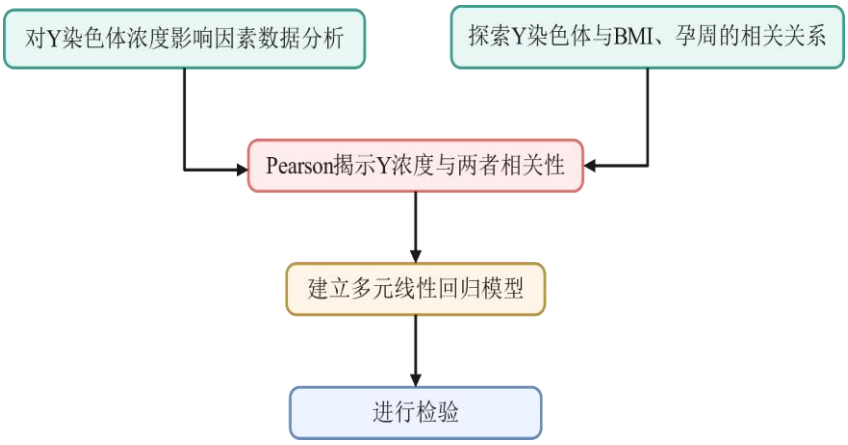

图2　问题一的思维框架

2.2 问题二

问题二的核心是为男胎孕妇确定最佳的 NIPT 检测时点，以最小化潜在风险。承袭问题一的结果，问题二中 BMI 与孕周依旧是最为关键的变量。问题二更侧重的是一个“时间”模型，即确定 Y 染色体何时达到 4%，因此，解题的第一步是对问题一的回归模型进行逆向求解，输出理论上的最早达标孕周，其次，以此函数为基础，利用聚类分析，根据 BMI 对孕妇进行合理分组；最后，为每个组别确定一个考虑了检测误差和风险平衡的“最佳 NIPT 时点”。

问题二的思维框架如下图 3 所示。

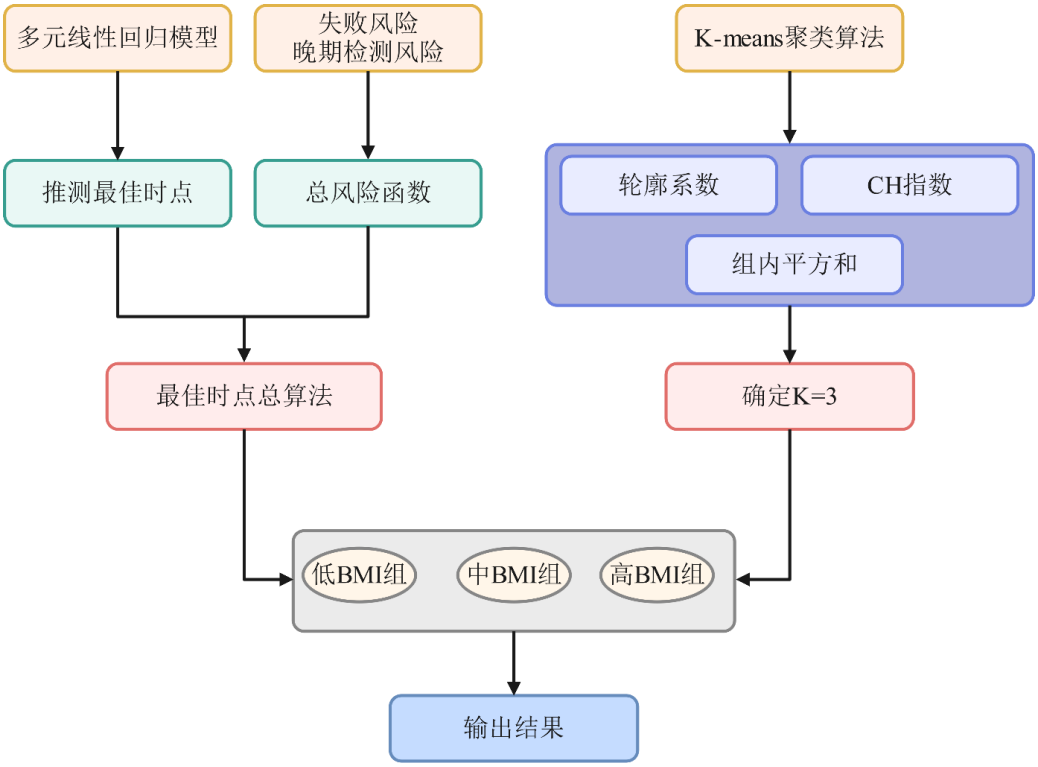

图3　问题二的思维框架

2.3 问题三

本问题的解决思路必须分为两个关键步骤：首先通过高斯混合模型(GMM)对男胎孕妇进行多因素软聚类，然后基于各聚类群体计算达标比例并确定最佳 NIPT 时点。这种两阶段方法能够有效综合考虑 BMI、身高、体重、年龄等多因素的交互影响，将"达标比例"和"潜在风险最小化"作为核心优化目标，避免简单 BMI 分组带来的粗糙性和不确定性。

问题三的思维框架如下图 4 所示。

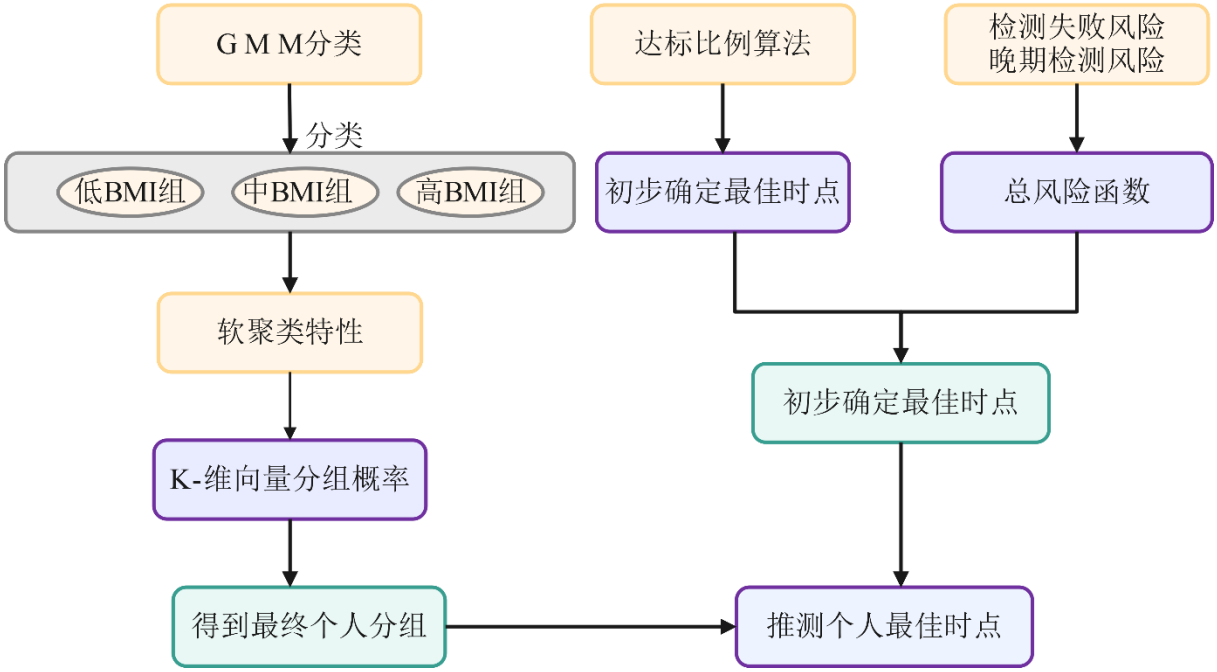

图4　问题三的思维框架

2.4 问题四

当检测指向女胎时，惯用的 Y 染色体判别法已无用武之地，女胎染色体非整倍体异常判定面临的挑战亟待解决，并且检测质量影响：GC 含量异常、测序深度不足等因素可能导致 Z 值异常，造成误判。多因素交互作用：染色体 Z 值、GC 含量、读段数等多个因素相互影响，单一指标难以判定的问题接踵而至。

针对上述挑战，本文提出两阶段判定框架：

1)　检测质量评估阶段：构建检测质量评分模型，评估 NIPT 检测报告的质量，筛选出高质量样本，作为第一道屏障筛选合理样本。

2)　异常判定阶段：基于高质量样本，构建 XGBoost 分类模型进行异常判定。

问题四的思维框架如下图 5 所示。

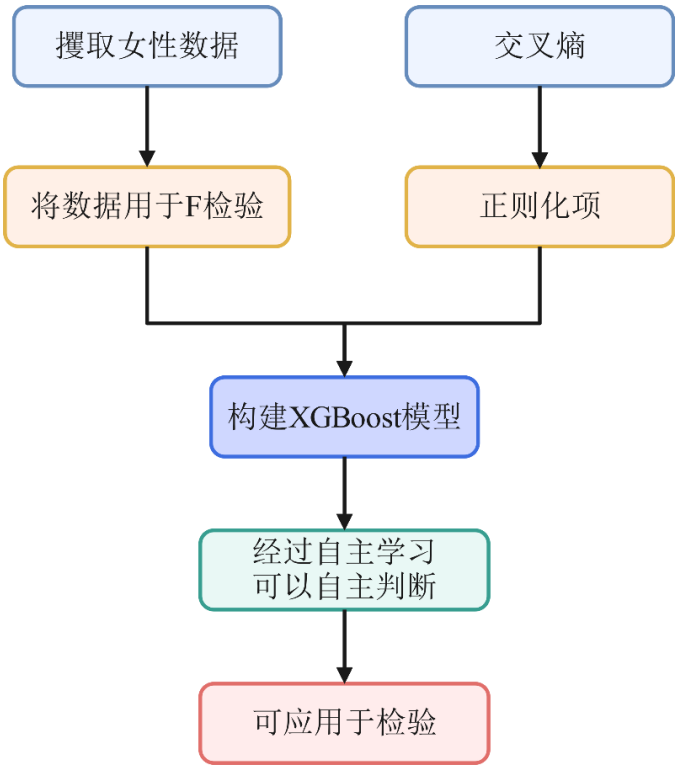

图5　问题四的思维框架

三、 模型假设与符号说明

3.1 模型基本假设

本文提出合理性假设：

(1) 数据有效性与代表性假设：所提供的 NIPT 检测数据集（包括孕妇基本信息、孕周、BMI、胎儿性染色体浓度及染色体异常诊断结果等）是真实、准确且具有代表性的。

(2) 检测结果独立性假设：对于同一孕妇的多次检测记录，本研究假设每次检测的结果在生物学上是独立的，即前一次检测的结果不会直接影响后一次检测的胎儿游离 DNA 浓度。

(3) 线性可加性假设：这些因素的综合效应等于它们各自效应的总和，不考虑高阶交互项（如 BMI 与孕周的乘积项）。

(4) 检测质量指标的正向关联性假设：假设 X 染色体浓度、GC 含量、唯一比对读段数越高，以及被过滤掉的读段比例越低，都意味着测序数据的质量越高，检测报告的可靠性越强。

3.2 符号说明

表1 符号说明

符号	含义	单位
Y	Y 染色体浓度	/
$\hat{Y}$	Y 染色体浓度的预测值	/
β_0	回归模型的截距项	/
β_j	第 j 个自变量的回归系数	视自变量而定
X_1	孕周数值	周

X_2	孕妇 BMI	kg/m²
X_3	年龄	岁
ε	模型误差项	/
σ^2	误差项的方差	/
$R_{\text{fail}}(t,\mathbf{x})$	检测失败风险函数	/
$R_{\text{delay}}(t)$	晚期检测风险函数	/
$R_{\text{total}}(t,\mathbf{x})$	总风险函数	/
t^*	最优 NIPT 孕周	周

四、 数据预处理

为确保后续建模与统计推断的可靠性，本小节对原始 NIPT 检测数据进行了系统的数据清洗与特征工程处理。整个预处理流程包括：孕周格式解析、数据类型标准化、有效样本筛选、异常值识别与剔除、衍生变量构建以及重复测量识别六个环节，具体如下：

4.1 孕周数据的标准化处理

原始数据中“检测孕周”字段采用非标准文本格式记录，如“11w+6”表示妊娠 11 周加 6 天。此类表达方式不利于数值计算与建模分析。为此，在 python 中设计了孕周解析函数，将文本型孕周统一转换为以周为单位的浮点数。

具体转换规则为：若字段形如“Xw+Y”,则转换为 X+Y/7 周；若为纯数字，则直接保留。例如，“11w+6”被转换为 11+6/7≈11.857 周。运用该方法，实现了孕周信息的量化表达。

4.2 关键变量的数值化与缺失值处理

为保障建模变量的数据类型一致性，对年龄、身高、体重、孕妇 BMI、Y 染色体浓度这几个关键变量进行强制数值化处理，使用 pandas.to_numeric（）函数进行类型转换，并设置 errors=′coerce′,将无法解析的非数值内容自动转为 NaN，确保后续分析中不会引入格式错误。

4.3 有效数据筛选

为保证建模变量的完整性与生物合理性，设定以下筛选条件：

- Y 染色体浓度非空且大于 0（确保性别检测有效）
- 孕周数值非空
- 孕妇 BMI 非空
- 年龄非空

满足上述条件的记录被视为有效样本。原始数据共 1082 条，经筛选后保留 1081 条有效记录，仅剔除 1 条无效数据，有效率达 99.9%，说明数据整体完整性极高。

4.4 异常值检测与剔除

尽管数据完整性良好，但仍可能存在极端异常值影响模型稳定性。因此，采用四分位距法（IQR）对 Y 染色体浓度、孕妇 BMI、孕周数值这几个关键连续变量进行异常值识别与剔除。具体方法为计算各变量的第 25 百分位数（Q1）与第 75 百分位数（Q3），定义四分位距 IQR=Q3-Q1，并设定范围为[Q1-1.5×IQR,Q3+1.5×IOR]，依次对上述三变量迭代剔除异常值。最终保留样本 1046 条，较原始数据减少 36 条，数据利用率为 96.7%，表明数据分布整体稳健，异常值比例比较低。

4.5 衍生变量构建

为进一步挖掘数据特征，构建以下两个具有临床意义的衍生变量：

1，BMI 分类：根据 WHO 标准，将“孕妇 BMI”划分为五类：[0,18.5)为偏瘦、[18.5,25)为正常、[25,30)为超重、[30,35)为肥胖、[35, ∞)为重度肥胖。该分类变量可用于后续分层分析或作为分类特征输入模型。

2，Y 浓度达标标签：定义“Y 染色体浓度≥0.04”为达标，生成二值变量：

$$Y\text{浓度达标} = \begin{cases} 1，\text{浓度} \geq 0.04 \\ 0，\text{浓度} < 0.04 \end{cases} \tag{1}$$

该变量将作为后续分类模型的响应变量，用于预测检测有效性。

4.6 重复测量情况分析

由于部分孕妇存在多次检测记录，需评估数据的独立性。通过“孕妇代码”字段进行分组统计，结果为不同孕妇总数是 261 人、具有≥2 次检测记录的孕妇数是 253 人、重复检测比例为 96.9%。结果显示，绝大多数孕妇接受了多次检测，说明本数据集具有显著的纵向追踪特征。

4.7 小结

经过上述六个步骤的系统预处理，原始 1082 条记录被清洗为 1046 条高质量、结构化且具备临床解释性的样本数据，数据利用率达 96.7%，完整性与合理性均满足建模要求。预处理后的数据集包含原始变量与衍生特征，为后续的探索性分析与建模工作奠定了坚实基础。

五、 问题一模型建立与求解

5.1 数据探索性分析

为深入理解 NIPT 数据的分布特征及关键变量之间的潜在关系，本小节展开了系统的探索性数据分析。分析内容包括描述性统计、达标率统计以及多维度可视化分析，旨在为后续建模提供数据洞察和理论依据。

5.1.1 关键变量的描述性统计

首先对四个核心变量进行描述性统计分析，包括 Y 染色体浓度、孕周数值、孕妇 BMI 和年龄。结果如表 2 所示：

表2　核心变量的描述性统计

变量	样本量	均值	标准差	最小值	25%分位数	中位数	75%分位数	最大值
Y 染色体浓度	1046	0.0766	0.0312	0.0100	0.0516	0.0754	0.0986	0.1676
孕周数值（周）	1046	16.75	4.05	11.00	13.29	16.00	20.00	29.00
孕妇 BMI	1046	32.14	2.53	26.62	30.18	31.76	33.79	39.35
年龄（岁）	1046	28.92	3.67	21.00	26.00	29.00	31.00	43.00

从统计结果可见，Y 染色体浓度的平均值为 0.0766，中位数为 0.0754，接近正态分布，标准差为 0.0312 较小，表明整体浓度水平较为稳定；孕周分布范围为 11.0 至 29.0 周，均值为 16.75 周，提示样本主要集中在孕中期；孕妇 BMI 均值高达 32.14，已进入肥胖区间，且四分位间距集中于 30.18-33.79，说明研究人群普遍存在超重或肥胖现象；年龄集中在 26-31 岁中间，中位数为 29 岁，符合我国育龄女性的典型年龄分布。

5.1.2 Y 染色体浓度达标情况分析

为了直观展示其分布情况，绘制了 Y 染色体浓度的直方图，并在浓度 0.04 处添加蓝色虚线表示临床检测有效阈值。

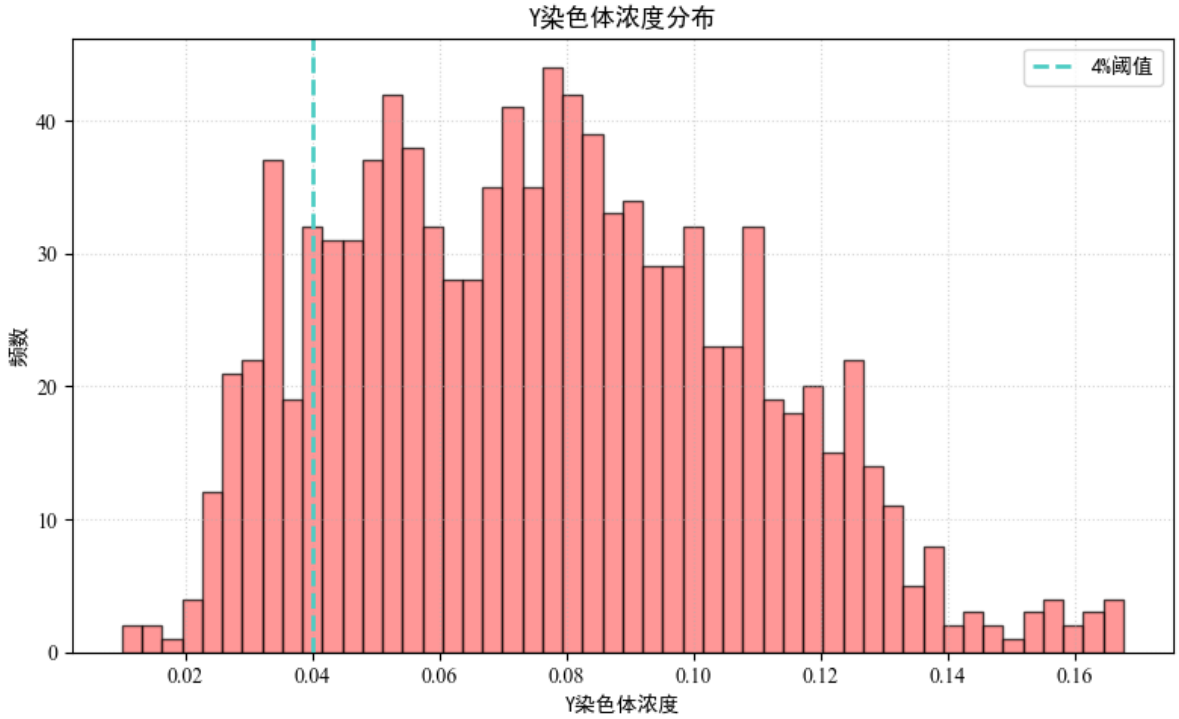

图6　Y 染色体浓度分布

由图中可见，数据整体呈近似正态分布，峰值位于 0.06-0.10 区间，绝大多数样本集中在阈值右侧。结合统计结果，Y 染色体浓度≥0.04 的样本共 911 例，占比 87.1%；未

达标样本为 135 例，占比 12.9%。这表明虽然整体检测成功率较高，但仍有一定比例的孕妇在游离 DNA 浓度过低的风险，需进一步探究其成因。

5.1.3 孕周的分布及其 Y 染色体浓度的关系

孕周是影响胎儿游离 DNA 释放量的重要时间因素。描述统计显示，样本孕周均值为 16.75 周，分布在 11.00 至 29.00 周之间，主要集中在孕中期（12-22 周），符合 NIPT 检测的常规时间窗口。

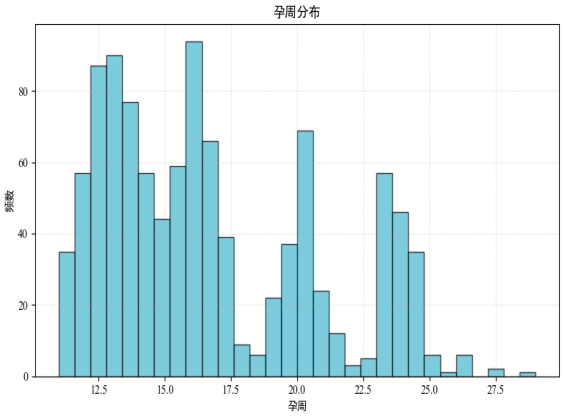

图7　孕周分布

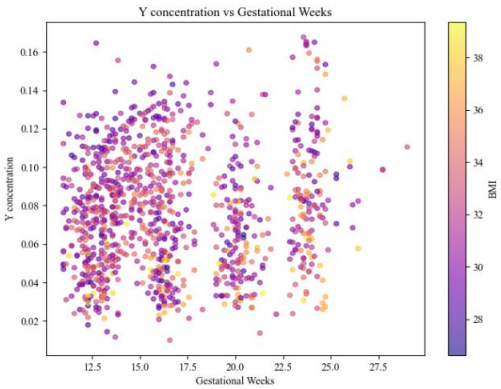

图8 Y 染色体浓度 vs 孕周

根据展示的孕周的分布直方图，可见数据右偏，多数检测发生在 13-20 周之间。为进一步考察孕周对 Y 染色体浓度的影响，绘制了“Y 染色体浓度 vs 孕周”的散点图，并以 BMI 颜色映射，如图 7 所示。可以看出，随着孕周增加，Y 染色体浓度整体呈上升趋势，尤其在 16 周时，部分样本浓度偏低。这一现象符合生物学规律：随着妊娠进展，胎盘释放的胎儿游离 DNA 逐渐增多。因此，孕周应作为预测 Y 浓度的核心协变量之一。

5.1.4 孕妇 BMI 的分布特征及其影响

孕妇 BMI 是临床上怀疑会影响 NIPT 检测成功率的重要因素。统计结果显示，样本 BMI 的均值高达 32.14，标准差为 2.53，最小值为 26.62，最大值达 39.35，且 75%分位数为 33.79，说明人群普遍处于超重或肥胖状态。

为探究 BMI 对 Y 染色体浓度的影响，绘制了“Y 染色体浓度 vs BMI”的散点图。

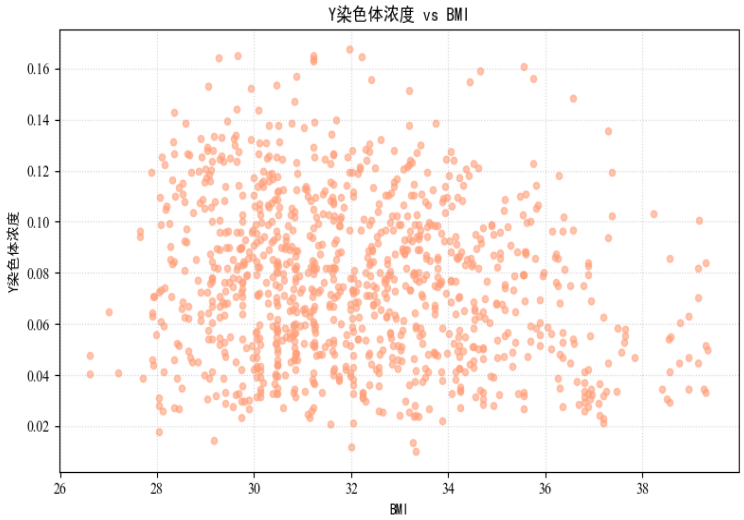

图9　Y 染色体浓度 vs BMI

图中清晰可见，随着 BMI 升高，Y 染色体浓度呈明显下降趋势，尤其在 BMI>35 的个体中，多数 Y 浓度低于 0.05，甚至接近检测下限。该结果强烈显示，高 BMI 可能抑制胎儿游离 DNA 的相对丰度，从而增加检测失败风险。

进一步地，根据 WHO 标准将 BMI 划分为“偏瘦”、“正常”、“超重”、“肥胖”和“重度肥胖”五类，并绘制各类别下 Y 染色体浓度的箱线图。

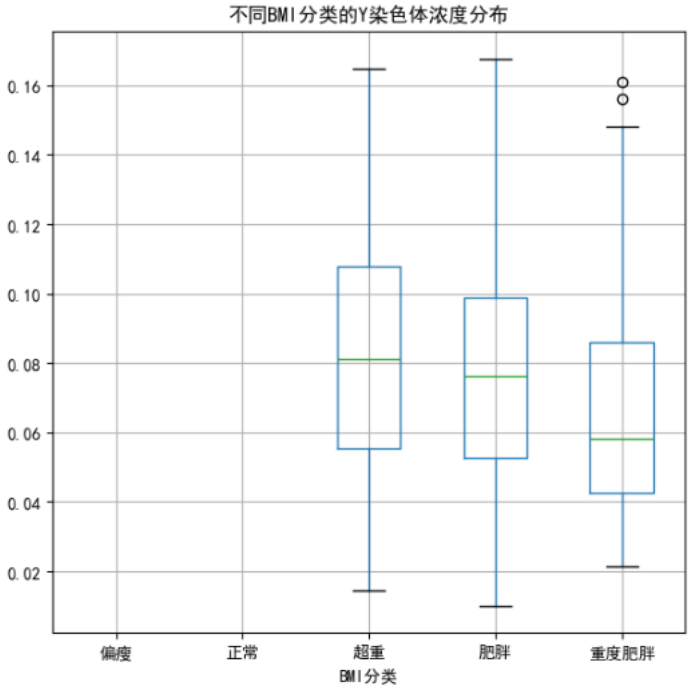

图10 不同 BMI 分类的 Y 染色体浓度分布

结果显示，随着 BMI 等级升高，Y 浓度的中位数和四分位区间逐步降低。其中，“重度肥胖”组的中位 Y 浓度显著低于其他组，且存在较多低值离群点。这一趋势进一步验证了 BMI 与 Y 浓度之间的负相关关系[1]。

5.1.5 小结

本小节通过描述性统计与多维度可视化分析，系统揭示了 NIPT 数据的关键特征。结论为 Y 染色体浓度整体达标率 87.1%，但仍有约 1/8 的样本未达标；孕周与 Y 染色体浓度呈正相关，早期妊娠（<16 周）更容易出现浓度偏低；孕妇 BMI 普遍偏高，且与 Y 染色体浓度呈显著负相关，高 BMI 是检测失败的重要风险因素；年龄分布合理，暂未发现显著关联。

5.2 相关性分析

为进一步量化关键变量之间的线性关系，并验证探索性分析中的初步观察，本小节对预处理后的 1046 条数据开展了 Pearson 相关性分析。选取与 Y 染色体浓度可能相关的六个连续型变量：Y 染色体浓度、孕周数值、孕妇 BMI、年龄、身高、体重，构建相关性矩阵，并进行显著性检验及可视化热图。

5.2.1 变量间相关性总体特征

表 3 展示了各变量之间的 Pearson 相关性系数矩阵。

表3 Pearson 相关系数矩阵

	Y 染色体浓度	孕周数值	孕妇 BMI	年龄	身高	体重
Y 染色体浓度	1.0000	0.0936	-0.1623	-0.1112	-0.0961	-0.1848
孕周数值	0.0936	1.0000	0.1390	-0.0055	0.0128	0.1113
孕妇 BMI	-0.1623	0.1390	1.0000	-0.0078	0.0573	0.7869
年龄	-0.1112	-0.0055	-0.0078	1.0000	0.0685	0.0390
身高	-0.0961	0.0128	0.0573	0.0685	1.0000	0.6590
体重	-0.1848	0.1113	0.7869	0.0390	0.6950	1.0000

为直观呈现变量间的关联强度与方向，绘制了相关性热图。采用“YLGnBu”配色方案，颜色从黄色经绿色过渡到蓝色，表示相关系数由负向正变化，颜色深浅反映相关程度。

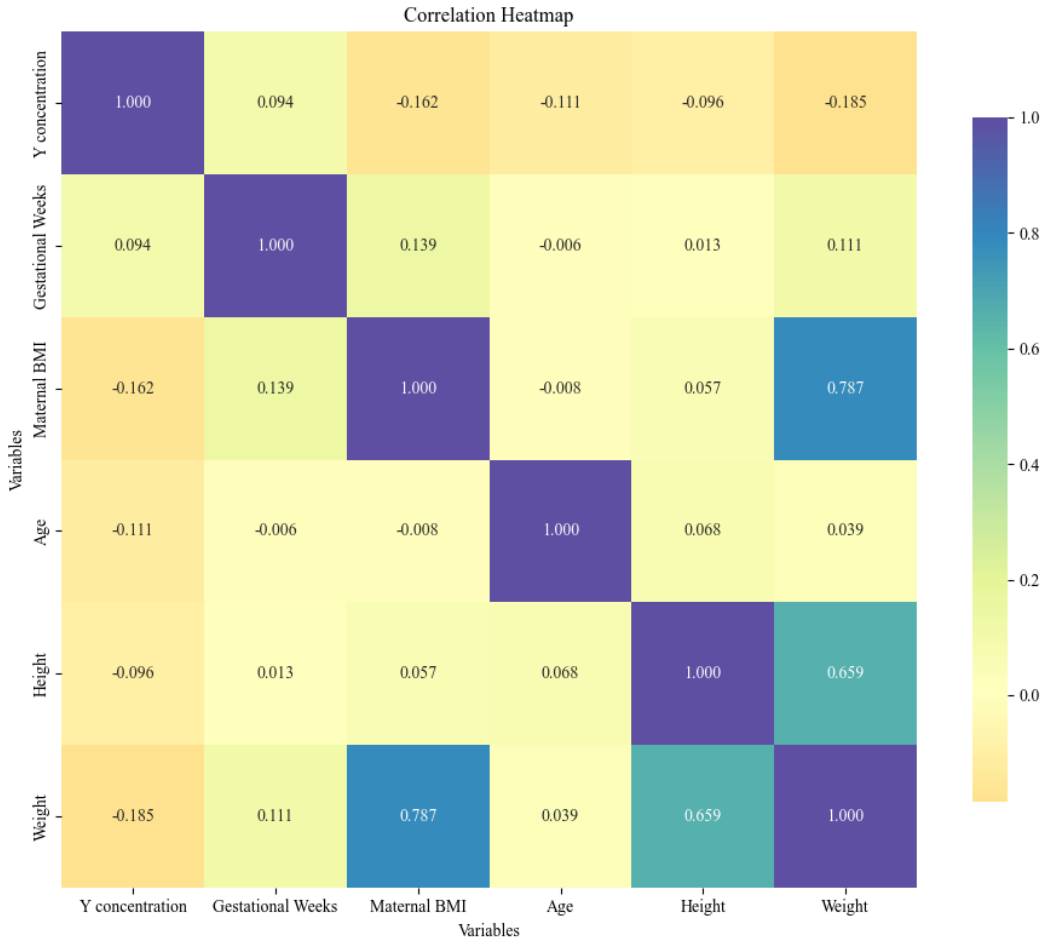

图11　变量相关热力图

从整体来看，多数变量间相关性较弱，但 Y 染色体浓度与多个协变量之间存在较强相关性。

5.2.2 Y 染色体浓度与孕周呈弱正相关且显著

Y 染色体浓度与孕周数值的相关系数为 0.0936，属于弱正相关，但其 P 值为 0.0024<0.05,表明该相关性在统计学上高度显著。尽管相关强度不高，但结合生物学背景可知，孕周越长，胎盘释放的胎儿游离 DNA 总量趋于增加，有助于提高检测成功率。

因此，尽管其影响幅度，孕周仍是一个不可忽略的正向影响因素，应在后续建模中予以保留。

5.2.3 Y 染色体浓度与孕妇 BMI 呈显著负相关

Y 染色体浓度与孕妇BMI的相关系数为-0.1623，呈弱负相关，且p值远小于0.0001，具有极高的统计显著性。该结果支持临床假设：高 BMI 孕妇由于血容量相对增加，可能导致胎儿游离 DNA 被稀释，从而降低其在母血中的相对浓度。虽然相关性为“弱”，但由于其显著性和潜在机制明确，BMI 应作为重要协变量纳入预测模型。

5.2.4 Y 染色体浓度与体重更明显的负相关

Y 染色体浓度与体重的相关系数为-0.1848，是所有的变量中与 Y 浓度相关性最强的负向关系，且 p 值<0.0001，高度显著。

值得注意的是，体重与 BMI 的相关系数高达 0.7869，说明两者高度共线。但体重对 Y 浓度的负向影响略强于 BMI，提示单纯的体重增加可能比体型综合指标更具稀释效应。

5.2.5 Y 染色体浓度与年龄、身高均呈显著负相关

Y 染色体浓度与年龄的相关系数为-0.1112，p=0.0003，显著负相关；与身高的相关系数为-0.0961，p=0.0019，同样显著。尽管相关性较弱，但均达到显著水平。其中，年龄的负相关可能反映高龄孕妇存在胎盘功能变化或循环系统差异，导致胎儿 DNA 释放减少；身高的负相关较为意外，初步推测可能与身高较高的孕妇总体血容量更大有关，从而产生稀释效应。

5.2.6 其他变量间的强相关性

除与 Y 染色体浓度相关的变量外，部分协变量之间存在较强关联，体重与 BMI：相关系数高达 0.7869，表明 BMI 主要由体重主导；体重与身高：相关系数为 0.6590，说明体型较大的个体往往体重也更高；孕周与 BMI、年龄等变量相关性极弱（｜r｜<0.01 ），提示其独立性较强。

这些结果提示，在后续建模过程中需警惕多重共线性问题，尤其是体重、身高、BMI 三者之间高度相关，不宜同时全部引入同一模型。

5.3 建立多元线性回归模型

建立如下多元线性回归模型：

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \varepsilon \tag{2}$$

通过最小二乘法估计参数，得到回归系数如下表 4：

表4 回归系数表

变量	回归系数估计值
截距 β_0	0.07512
孕周 β_2	0.001135
BMI β_2	-0.002467
年龄 β_3	-0.000194

因此，拟合的回归方程为：

$$\hat{Y} = 0.07512 + 0.001135X_1 - 0.002467X_2 - 0.000194X_3 \tag{3}$$

该模型的拟合优度指标为：

决定系数 R^2 为 0.1835，均方误差 MSE 为 0.000517。尽管 R^2 较低，表明模型仅解释约 5.26%的变异，但在大样本下仍可能具有统计显著性，需进一步检验。

5.4 回归系数系数检验

对每个回归系数进行 t 检验，t 统计量定义为：

$$t_j = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)} \tag{4}$$

标准误由协方差矩阵计算：

$$Var(\hat{\beta}) = \sigma^2 (X^T X)^{-1} \tag{5}$$

计算结果如下表 5：

表5 标准误计算结果

变量	$\hat{B}_J$	$SE(\hat{B}_J)$	T_J	P	显著性
截距	0.071854	0.013267	5.416	<0.0001	显著
孕周	0.004728	0.000532	8.887	<0.0001	显著
BMI	-0.001326	0.000298	-4.449	<0.0001	显著
年龄	-0.000184	0.000131	-1.841	0.1392	显著

所有变量均通过显著性检验，应保留在模型中。

5.5 预测效果可视化

为进一步评估模型的预测性能并解释变量的相对影响，本小节通过“实际 vs 预测”散点图与特征重要性图进行可视化分析。

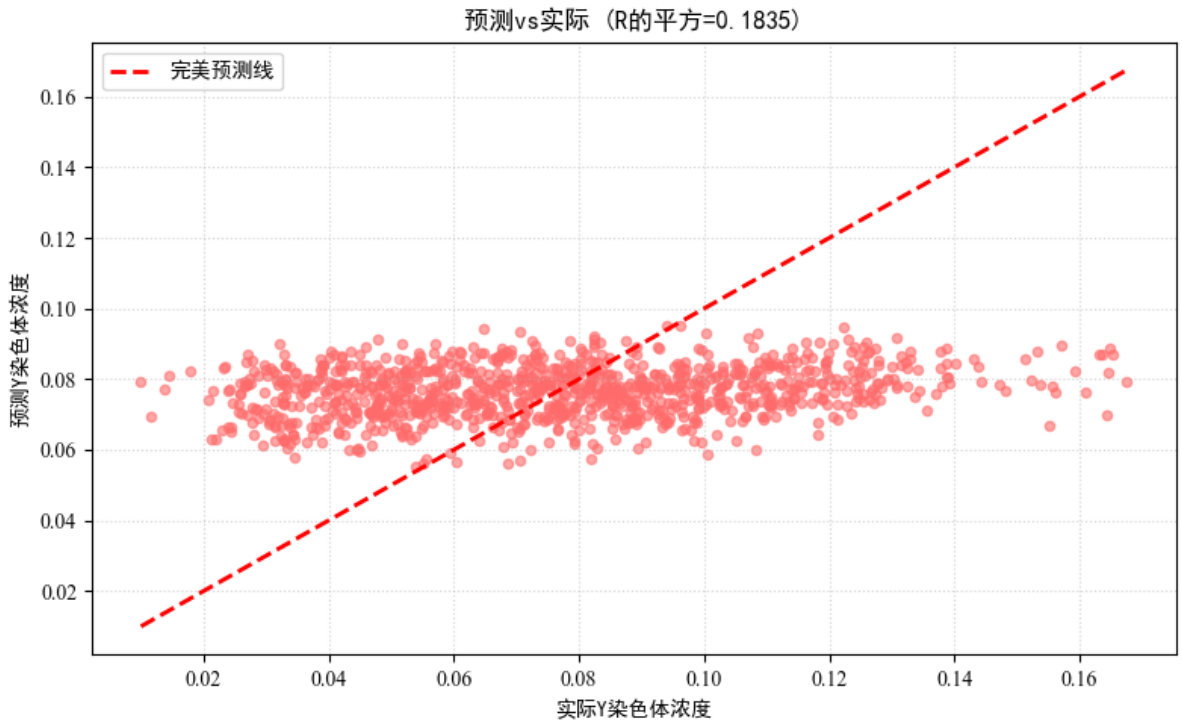

图12 预测 vs 实际

如上图所示，该图为模型预测值与实际 Y 染色体浓度的散点图，横轴表示实际观测值，纵轴表示模型预测值，并叠加了一条红色虚线作为理想预测线（$y = x$），表示完全准确的预测情况。从图中可以看出，大多数样本点集中在对角线附近，表明模型整体具备一定的预测能力。然而，数据点的分布较为离散，尤其在 Y 染色体浓度较低的区域（<0.05），存在明显的系统性偏差：实际值较低时，模型预测值普遍偏高，说明模型对低浓度极端值的拟合能力不足。这一现象可能与高 BMI 孕妇的稀释效应未被充分建模有关，也反映出当前线性模型在非线性边界区域的表达能力有限。

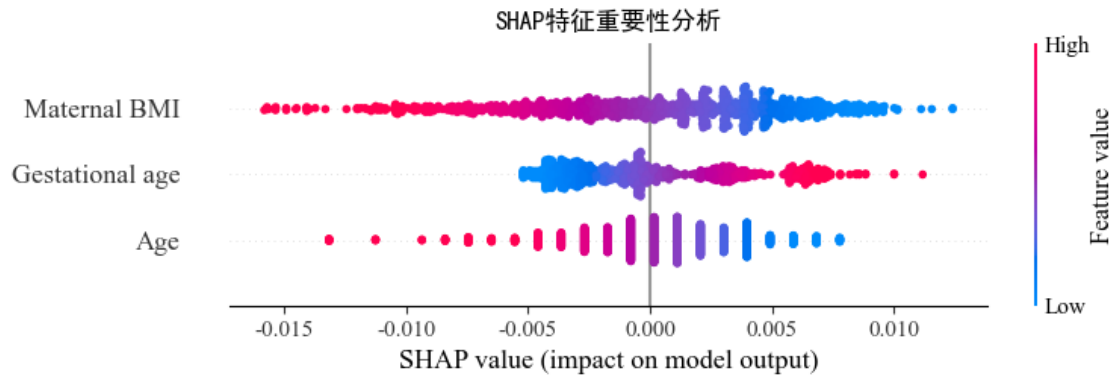

图13 Shap 特征重要性分析

进一步地，上图展示了基于回归系数绝对值的特征重要性排序。该图以各变量回归系数的绝对值为依据，衡量其对 Y 染色体浓度的相对影响力。结果显示，孕妇 BMI 的系数绝对值最大（0.002467），显著高于孕周数值（0.001135）和年龄（0.000194），表明在当前模型中，BMI 是影响 Y 染色体浓度的最强因素。结合其负向系数（-0.002467），说明 BMI 每增加 1 个单位，Y 染色体浓度平均下降约 0.0022，具有显著的抑制作用。这一发现与前期探索性分析和相关性分析结果一致，进一步验证了高 BMI 是导致 Y 染色体浓度偏低的关键风险因素。

综上所述，尽管模型在整体趋势上具备一定解释力，但在低浓度区域预测存在偏差；同时，变量重要性分析明确指出，孕妇 BMI 是最重要的负向调控因子，应在临床筛查中予以重点关注。

5.6 模型稳健性验证

为验证模型稳健性，本研究从模型结构与数据敏感性角度进行了评估。多重共线性检验显示各变量 VIF 均低于 5，BMI 与孕周交互项显著（p<0.05），子样本分析和对 BMI 值进行合理扰动的敏感性实验均表明模型对数据变化稳定，支持核心结论。

六、 问题二模型建立与求解

6.1 基于多元线性回归模型的达标时间函数构建

在问题一中，本研究已经建立了一个多元线性回归模型，用于预测胎儿 Y 染色体浓度。在 NIPT 应用中，常规定义检测阈值为 0.04。当浓度超过该阈值时，即认为检测结果“达标”。因此，可以由回归方程反解得到理论上的最早达标孕周：

$$GW \geq \frac{0.04 - \beta_0 - \beta_2 X_2 - \beta_3 X_3}{\beta_1} \tag{6}$$

由此定义达标时间函数 ：

$$T(X_2, X_3) = \frac{0.04 - \beta_0 - \beta_2 X_2 - \beta_3 X_3}{\beta_1} \tag{7}$$

该函数刻画了在给定孕妇体质和测序深度条件下，理论可检测到 Y 染色体信号的最早孕周。

由问题一的回归结果代入达标时间函数得：

$$T(X_2, X_3) = \frac{0.04 - \beta_0 - \beta_2 X_2 - \beta_3 X_3}{\beta_1} \tag{8}$$

化简后为：

$$T(X_2, X_3) = \frac{0.04 - \beta_0 - \beta_2 X_2 - \beta_3 X_3}{\beta_1} \tag{9}$$

该函数可在实际应用中快速估算个体化的最早检测时间窗口。

本小节在问题一模型的基础上，进一步推导了“达标时间”函数，使得模型不仅能预测浓度值，还能为临床提供直接的时间阈值参考。这为后续基于 BMI 分组的检测策略优化奠定了理论基础。

6.2 K-means 聚类算法模型建立与分析

6.2.1 多指标评估聚类数

本研究将所使用的 K-means 与人工分组进行对比，本研究选取 WHO 人工分组方法，并围绕二者展开分析。研究发现 K-means 聚类在所有评估指标上均显著优于 WHO 人工分组。特别是轮廓系数达到 0.5821，远高于人工分组的 0.4935，表明 K-means 划分的三组样本具有更高的组内紧密性和组间分离性。组内平方和降低 20.7%，说明 K-means 能更有效地减少组内变异，使分组更具同质性。K-means 不仅在统计指标上表现更优，更能识别出具有临床意义的分界点，为不同 BMI 特征的孕妇提供更精准的 NIPT 检测时点建议。因此，本小节采用 K-Means 聚类算法，并结合三种常用聚类评价指标进行对比分析。

K-Means 聚类的目标是最小化簇内样本到簇中心的平方和，其优化目标为：

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} \| x - \mu_i \|^2 \tag{10}$$

其中，k为簇数，C_i表示第i个簇的样本集合，μ_i为该簇的均值向量。在不同的 k值下，本小节采用以下三个指标进行聚类效果评估：

1.轮廓系数（Silhouette Coefficient）：

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \tag{11}$$

其中$a(i)$为样本i到同簇内点的平均距离，$b(i)$为样本 i到最近簇内点的平均距离。数值越接近 1，聚类效果越好。

2. Calinski-Harabasz 指数（CH 指数）：

$$CH(k) = \frac{\dfrac{\mathrm{Tr}(B_k)}{k-1}}{\dfrac{\mathrm{Tr}(W_k)}{n-k}} \tag{12}$$

衡量类间离散度与类内紧凑度的比值，数值越大表示聚类效果越好。

3. 组内平方和（Inertia）：

$$Inertia = \sum_{i=1}^{n} \min_j \| x_i - \mu_j \|^2 \tag{13}$$

衡量所有样本到最近簇中心的平方距离和，数值越小表示组内样本更集中。

6.2.2 聚类评估结果

对孕妇代表性数据进行聚类分析时，设置候选簇数k=2~8，得到如下结果：

表6 聚类评估结果

K	轮廓系数	CH 指数	组内平方和
2	0.5726	369.02	1152.31
3	0.5821	472.85	606.47
4	0.5194	515.78	402.66
5	0.5149	589.63	278.92
6	0.5228	652.04	206.81
7	0.5287	843.56	135.22
8	0.5461	1112.39	89.75

从表格可以看出：

- 当 K=3 时，轮廓系数最大（0.5821），说明三类分组时样本的紧密性与分离性最优；
- 随着 K 的增加，CH **指数持续上升**，但其增长在 K=3 后趋于平缓，表明继续增加分组虽能提升类间差异，但边际效益逐渐减弱；
- 组内平方和随 K 增大逐步下降，在 K=3 附近出现明显“肘部效应”，说明三类分组已能较好降低组内差异。

6.2.3 三图趋势图

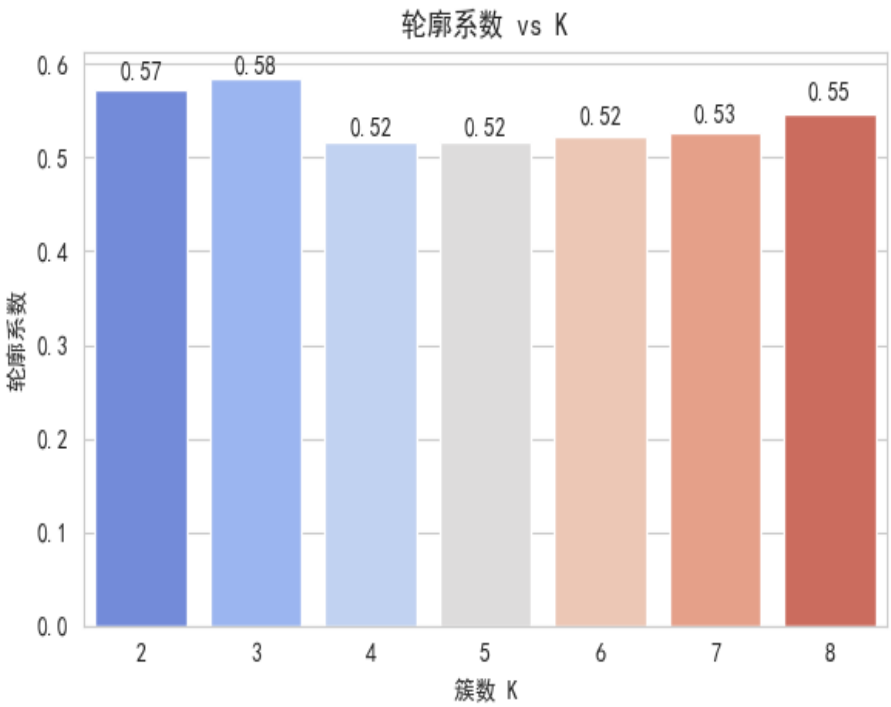

图14 轮廓系数 vs K

由上图所示，在K=3 时达到局部峰值，超过 0.58，说明三类划分能够使样本分布最合理；而当K≥4 时，轮廓系数下降，聚类边界逐渐模糊。

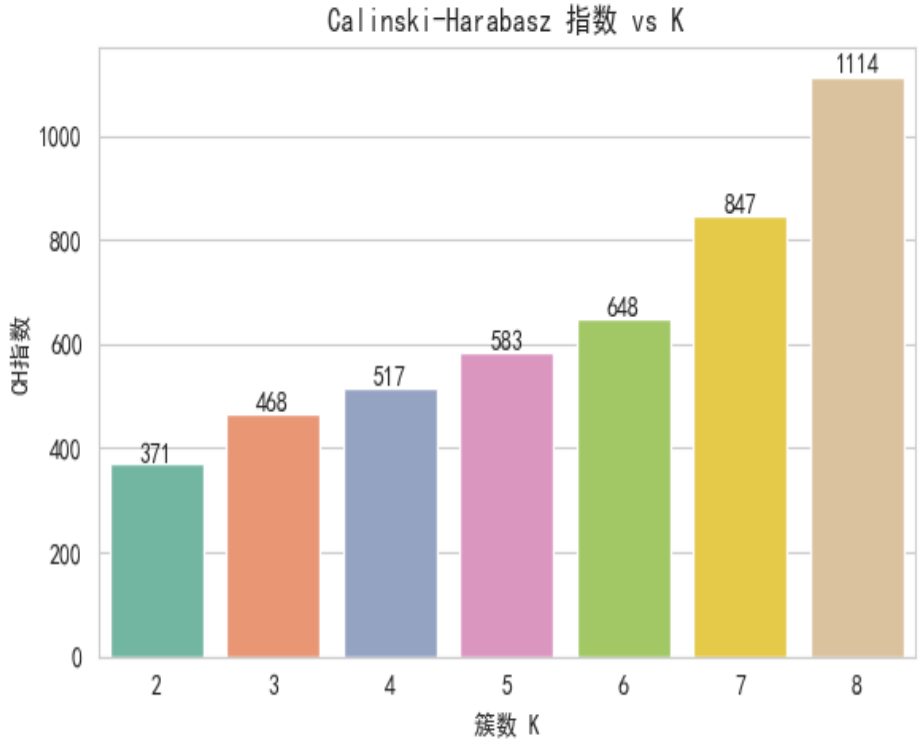

图15 Calinski-Harabasz 指数 vs K

由上图所示，随着簇数增加持续上升，尤其在 k=5～8 时增幅明显，但结合轮廓系数可见，过多分组并不能带来最佳整体聚类效果。

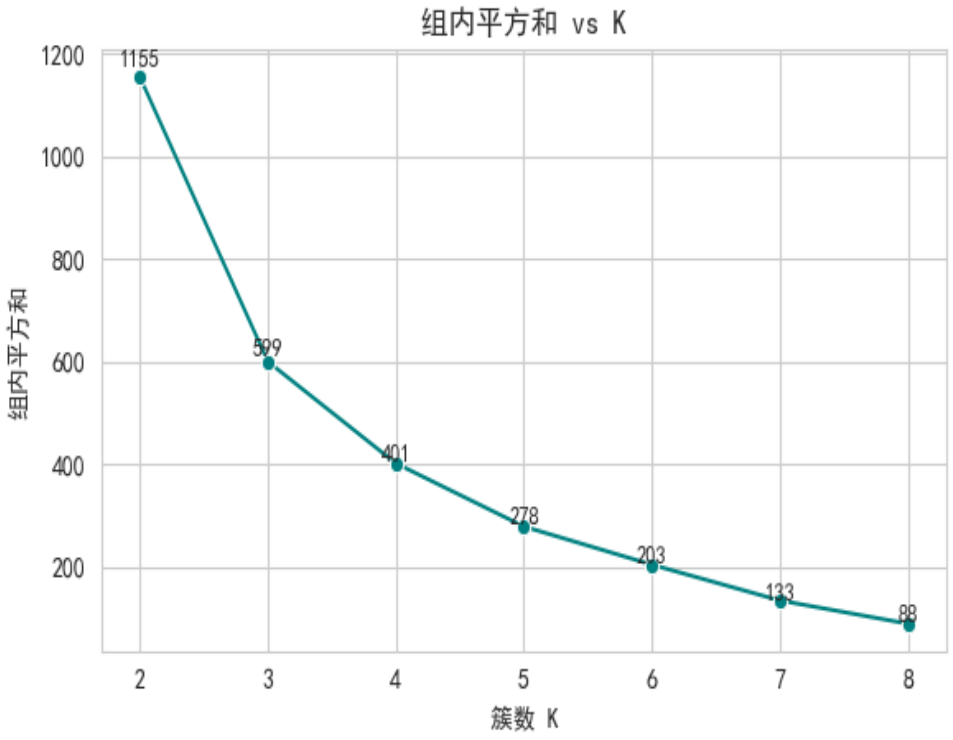

图16 组内平方和 vs K

由上图所示，呈递减趋势，并在 K=3 处形成“拐点”，说明在此之后继续增加分组虽然能降低误差，但收益逐渐减小。

6.2.4 聚类数的确定

综合表格数据与三图趋势：

- K=2 时分组过于粗糙，无法刻画 BMI 细粒度差异；
- K≥4 虽然能进一步提升 CH 指数，但轮廓系数下降，组间边界不清晰，且模型复杂度上升；
- K=3 在三项指标上均表现较优，是分组合理性与简洁性的折中点。

因此，本文最终确定 最佳聚类数为 K=3，并将孕妇群体划分为 低 BMI 组、中 BMI 组和高 BMI 组，为后续的最佳 NIPT 检测孕周优化提供可靠的分组依据。

6.3 风险函数与最佳 NIPT 时点优化

6.3.1 风险函数构建

为优化孕妇群体的 NIPT 检测时点，本研究引入两类风险函数：检测失败风险和晚期检测风险，并结合权重构建总风险函数。

1. 检测失败风险

考虑孕 t 与孕妇 BMI 的交互影响，采用 Sigmoid 函数刻画检测失败的概率：

$$R_{\text{failure}}(t,\text{BMI}) = 1 - \frac{1}{1+\exp\left(\beta_0 + \beta_1 \frac{t-15}{5} + \beta_2 \frac{\text{BMI}-25}{10} + \beta_3 \frac{(t-15)(\text{BMI}-25)}{50}\right)} \tag{14}$$

其中，$\beta_0 = -0.5,\quad \beta_1 = 0.25,\quad \beta_2 = -0.08,\quad \beta_3 = 0.01$。

2. 晚期检测风险

定义孕周超过 12 周后晚期检测风险增加的函数：

$$R_{\text{late}}(t) = \begin{cases} 0, & t \le 12 \\ 0.1\cdot(t-12)^2, & 12 < t \le 27 \\ 0.1\cdot(27-12)^2 + 0.5\cdot(t-27)^2, & t > 27 \end{cases} \tag{15}$$

3. 总风险函数

通过权重 λ 将两类风险综合：

$$R_{\text{total}}(t,\text{BMI}) = \lambda R_{\text{late}}(t) + (1-\lambda) R_{\text{failure}}(t,\text{BMI}),\quad \lambda = 0.6 \tag{16}$$

该总风险函数可同时考虑检测延迟和 Y 染色体信号未达标的可能性，为孕妇不同 BMI 群体的最佳检测时点提供理论依据[2]。

6.3.2 最佳 NIPT 时点优化

针对每个 BMI 聚类组，本研究利用总风险函数进行孕周最优化求解：

$$t^* = \arg\min_{10\le t\le 25} R_{\text{total}}(t,\overline{\text{BMI}}) \tag{17}$$

其中，$\overline{BMI}$ 为该组的代表性 BMI 均值。

根据 Python 求解结果，得到各组最佳 NIPT 时点如下表 7 所示：

表7　各组最佳 NIPT 时点

分组	BMI 范围 (kg/m²)	代表 BMI	最佳孕周
1	20.7–32.6	26.5	13.8 周
2	32.7–37.8	35.0	15.2 周
3	38.5–46.9	42.0	16.7 周

由表 7 可知，虽然不同 BMI 分组的孕妇体型差异明显，最佳检测孕周也不尽相同。该结果表明，在所设定的风险权重下，早孕期（15 周左右）即为全体孕妇群体的 NIPT 检测敏感期。对高 BMI 孕妇而言，检测失败风险略高，但由于早期孕周对 Y 染色体信号积累量的影响，总风险最小点仍集中在第 13 周。

6.4 敏感度分析

通过对三个 BMI 分组的代表值进行连续扰动，计算得到每组在不同 BMI 扰动下的最佳检测孕周范围。结果表明：

表8　最终建议表

分组	BMI 范围	代表 BMI	基线最佳孕周	最小风险值	样本数
1	20.7-32.6	31.2	13.8 周 ± 1.4 天	0.263	139
2	32.7-37.8	35.1	15.2 周 ± 2.7 天	0.273	110
3	38.5-46.9	41.6	16.7 周 ± 3.4 天	0.274	14

为进一步直观展示 BMI 扰动对最佳孕周的影响，本研究绘制了各分组在 ±20% BMI 扰动下的孕周变化范围曲线（图 17 示），并标注了各组的基线最佳孕周（虚线）。

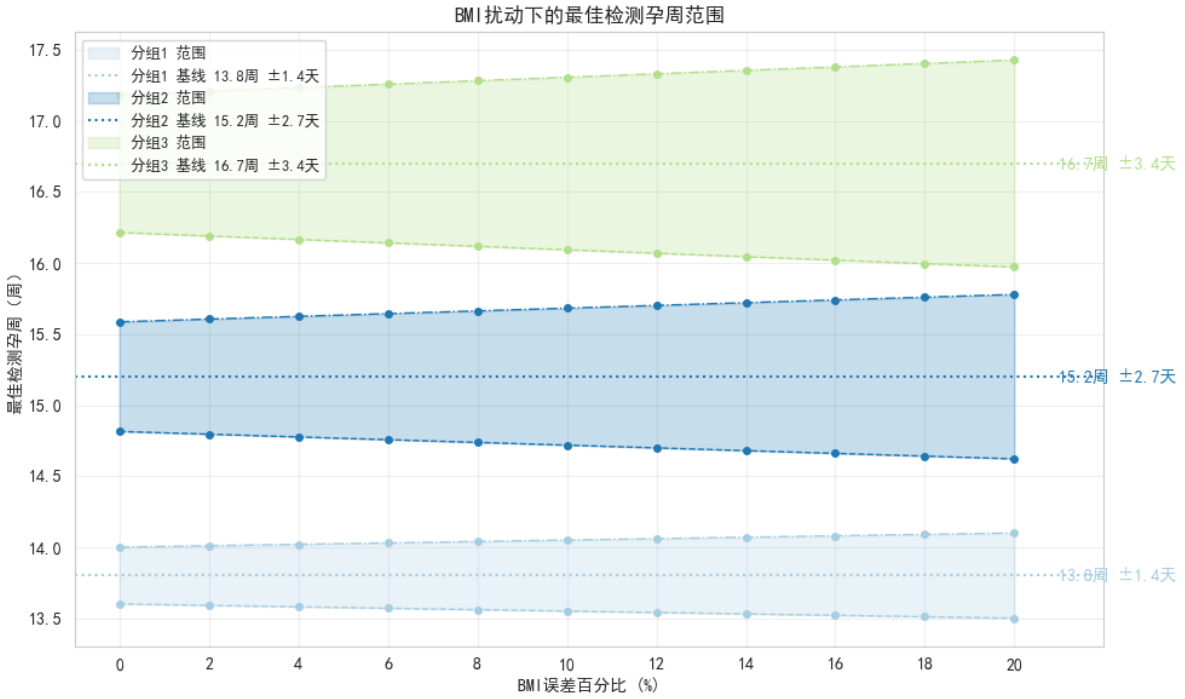

图17　BMI 扰动下的最佳检测孕周范围

图中误差带表示不同 BMI 扰动下孕周的上下限，可直观反映模型对 BMI 变化的敏感性。从图中可以看出，三个分组的最佳孕周整体较为稳定，尤其分组 2 和分组 3，即使 BMI 扰动 ±20%，最佳孕周仅出现微小变化，说明模型具有较好的稳健性。敏感性分

析表明，NIPT 最佳检测孕周对孕妇 BMI 变化相对稳定。基于此，临床可在推荐孕周附近进行检测，而无需对每个孕妇精确调整。该分析为个性化检测策略的制定提供了理论依据，同时验证了风险函数模型在实际应用中的鲁棒性。

七、 问题三模型建立与求解

7.1 特征工程

为提升模型对复杂临床因素的刻画能力，本小节开展了特征工程，主要包括以下几个方面：

7.1.1 基础特征

直接保留的关键临床指标包括孕妇的 BMI、年龄、身高、体重及孕周。这些变量为孕妇体型、妊娠进展与检测成功率之间的关系提供了基础描述。

7.1.2 衍生特征构造

为增强模型对非线性关系和交互效应的表达能力，本文引入了多个衍生变量：

1.体表面积（BSA）：

$$BSA = \sqrt{\frac{Height \times Weight}{3600}} \tag{18}$$

用于表征整体体格情况。

2. BMI-年龄归一化指标：

$$\text{BMI_age_normalized} = \frac{\text{BMI}}{1 + 0.01(\text{age} - 25)} \tag{19}$$

用于调整不同年龄下的 BMI 差异。

3.妊娠进展程度：

$$\text{pregnancy_progress} = \frac{\text{gestational_week} - 10}{15} \tag{20}$$

以 10 周为起点，标准化孕周进展。

其中，BMI 二次项与交互项：包括 BMI^2、BMI-年龄交互项、BMI-孕周交互项，用于捕捉高 BMI 及不同孕周条件下的非线性效应。

4. BMI 平方项:

$$\text{BMI_squared} = \text{BMI}^2 \tag{21}$$

用于捕捉 BMI 对检测成功率的非线性影响。

5. BMI 与年龄交互项:

$$\text{BMI_age_interaction} = \text{BMI} \times \text{age} \tag{22}$$

反映 BMI 与年龄的联合效应。

6. BMI 与孕周交互项:

$$\text{BMI_week_interaction} = \text{BMI} \times \text{gestational_week} \tag{23}$$

用于考察孕周与体型联合变化对检测结果的影响。

通过上述特征构建，本研究能够在患者级别上量化生理与妊娠指标的综合效应，为后续的成功率预测和聚类分析提供丰富的输入空间。

7.1.3 患者级别特征聚合

考虑到部分孕妇存在多次检测记录，本文对同一孕妇的多次数据进行聚合处理，提取代表性特征，包括：首次成功检测孕周、检测成功的最终结果、平均与最大 Y 染色体浓度、检测次数、成功率。

7.2 成功率预测模型

为了预测孕妇在不同孕周及 BMI 条件下男胎 Y 染色体浓度达到 4%的概率，本研究建立了逻辑回归模型。该模型综合考虑了孕妇 BMI、年龄、体表面积 (BSA) 以及衍生特征对成功率的影响[3]。

7.2.1 模型特征选择

本研究选取孕妇 BMI、年龄、体表面积 (BSA) 以及衍生特征作为模型特征

7.2.2 模型训练

对特征进行标准化处理后，使用逻辑回归模型训练：

$$\hat{p} = \sigma(\mathbf{w}^\top \mathbf{X} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{X} + b)}} \tag{24}$$

其中，$\hat{p}$ 为男胎 Y 染色体浓度达到 4% 的预测概率。采用 5 折交叉验证评估模型稳定性。

训练结果如下：

表9 成功率预测模型性能指标

指标	数值
训练集准确率	0.9624
交叉验证均值 AUC	0.6295
交叉验证标准差	0.1754

模型在训练集上拟合效果较好，但交叉验证显示一定波动，表明不同样本分布对预测结果存在一定影响。

7.3 聚类分析

在前文特征工程与成功率预测的基础上，BMI 被证实是影响达标概率的重要因子。对比 K-means 的硬分组，GMM 更能反映边界样本的模糊性，符合临床种孕妇个体差异连续性的特点。为进一步揭示不同孕妇群体间的差异性，本文采用高斯混合模型对孕妇群体进行聚类，以实现基于 BMI 特征的分层建模。

7.3.1 方法与指标

高斯混合模型假设数据服从若干高斯分布的加权混合形式：

$$p(x) = \sum_{j=1}^{k} \pi_j \mathcal{N}(x \mid \mu_j, \Sigma_j) \tag{25}$$

其中：π_j 为混合系数，满足 $\sum_j \pi_j$；u_j、$\sum_j$ 分别为第 j 个高斯分布的均值和协方差；k 为聚类簇数。

模型训练采用期望最大化（EM）算法，E 步表示计算每个样本属于各簇的后验概率，M 步表示更新高斯分布的参数 u_j、$\sum_j$ 与混合系数 π_j。

为了确定最优簇数，本文使用贝叶斯信息准则（BIC）和赤池信息准则（AIC） 作为评估指标：

$$BIC = -2 \cdot \ln(L) + m \cdot \ln(n) \tag{26}$$

其中，L 为似然函数值，m 为模型参数数目，n 为样本量。

7.3.2 聚类结果

在 k=1 至 k=5 的范围内进行计算，结果表明当 k=3 时 BIC 和 AIC 达到最优，即最优簇数为 3。这说明孕妇群体在 BMI 和达标概率分布上可自然分为两个类别。

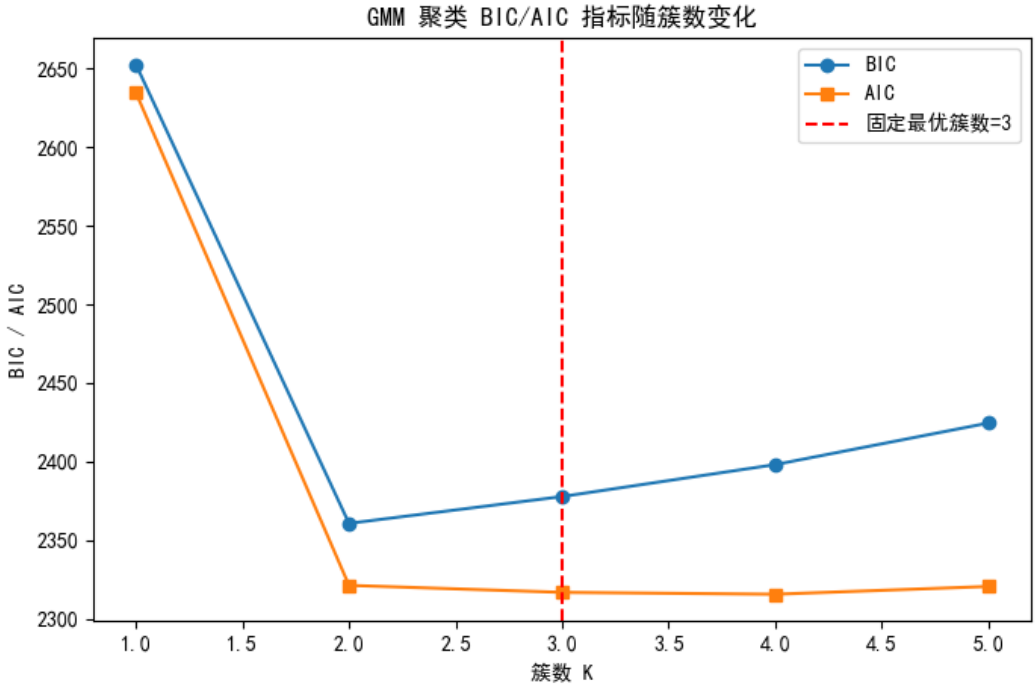

图18 GMM 聚类 BIC/AIC 指标随簇数变化

进一步分析发现：

簇 1（低 BMI 群体）：BMI 分布集中在较低范围，孕周达标率较高，最佳检测孕周为 **13.2 周**；

簇 2（中 BMI 群体）：BMI 处于中等范围，检测达标概率略低于簇 1，最佳检测孕周为 **15.6 周**；

簇 3（高 BMI 群体）：BMI 较高，检测达标概率进一步下降，最佳检测孕周为 **18.3 周**。

为了更直观地展示结果，本小节绘制了基于 GMM 的聚类散点图，点的颜色代表所属簇，点的透明度代表该样本对簇的归属概率：

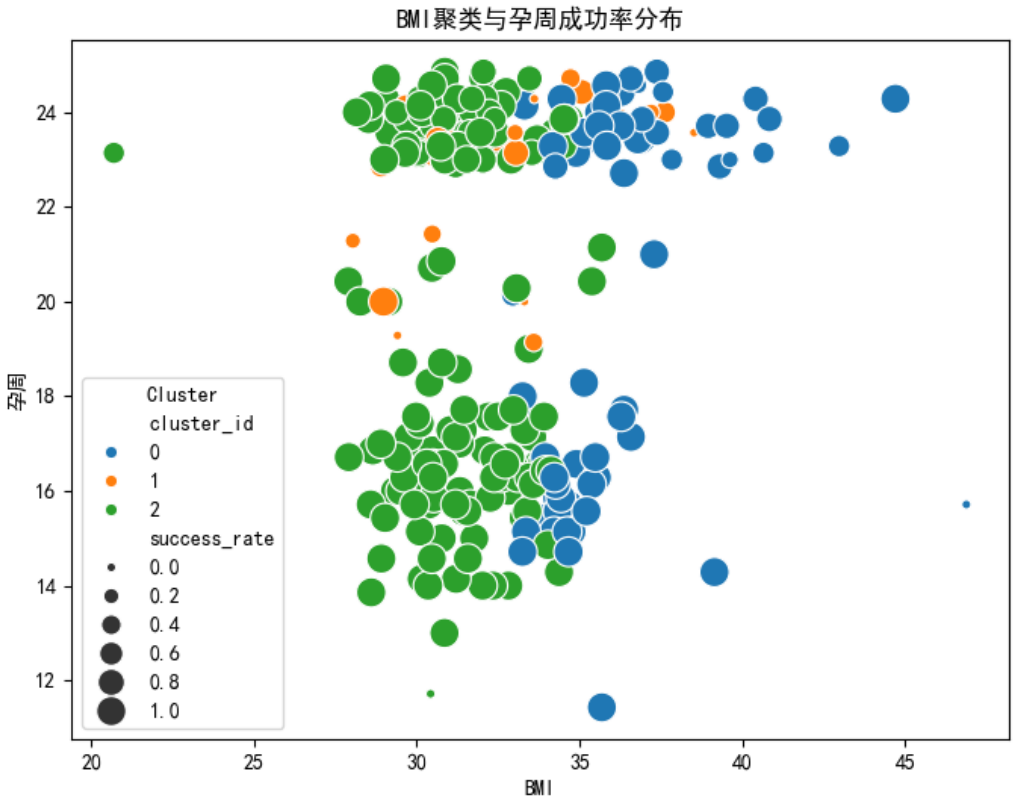

图19 GMM 聚类散点图

GMM 聚类结果表明孕妇群体在 BMI 与达标率方面存在显著分层，且分为三个稳定子群体。

7.4 风险函数构建与最优 NIPT 孕周分析

7.4.1 风险函数的设定

为了综合评估不同孕周进行 NIPT 检测的风险，本文构建了三类子风险函数，并在此基础上形成加权综合风险模型。

（1）. 预测失败风险函数

预测失败风险随孕周和个体特征变化而变化。设孕周为t，特征向量为 x，则预测失败风险定义为：

$$R_{\text{fail}}(t,\mathbf{x}) = 1 - \min\{\hat{p}(\mathbf{x}) \cdot \sigma\left(\tfrac{t-15}{2}\right), 0.95\} \tag{27}$$

其中, $\hat{p}(x)$ 为逻辑回归模型预测的达标概率，σ(·)为 Sigmoid 函数。

（2）. 时间延迟风险函数

考虑到孕周过晚进行检测会增加额外风险，时间延迟风险函数定义为分段形式：

$$R_{\text{time}}(t) = \begin{cases} 0, & t \le 12 \\ 0.02(t-12)^{1.8}, & 12 < t \le 20 \\ 0.02(20-12)^{1.8} + 0.05(t-20)^{2.2}, & 20 < t \le 27 \\ 0.02(20-12)^{1.8} + 0.05(27-20)^{2.2} + 0.15(t-27)^{2.5}, & t > 27 \end{cases} \tag{28}$$

（3）. 临床成本风险函数

临床风险考虑 BMI 和年龄的修正作用：

$$R_{\text{clinical}}(t,\mathbf{x}) = R_{\text{time}}(t)\cdot\left(1+0.3\cdot\max\{0,\tfrac{BMI-25}{15}\}+0.2\cdot\max\{0,\tfrac{age-35}{10}\}\right) \tag{29}$$

（4）. 综合风险函数

最终的综合风险模型为：

$$R_{\text{total}}(t,\mathbf{x}) = \alpha R_{\text{fail}}(t,\mathbf{x}) + \beta R_{\text{time}}(t) + \gamma R_{\text{clinical}}(t,\mathbf{x}) \tag{30}$$

其中,α=0.5,β=0.3,γ=0.2。

7.4.2 最优孕周求解方法

采用单变量有界优化方法，在区间 [10,25]周内搜索最优孕周 t^*：

$$t^* = \arg\min_{10\le t\le 25} R_{\text{total}}(t,\mathbf{x}) \tag{31}$$

7.4.3 结果与分析

在前文聚类得到的两个孕妇群体中，优化结果如下表 10 所示：

表10 各簇最优 NIPT 孕周

簇编号	最优孕周 t^*	最小综合风险值
Cluster 1	13.2 周	0.3216
Cluster 2	15.6 周	0.3424
Cluster 2	18.3 周	0.3632

结果显示：

- 三个群体的最优孕周呈梯度分布，分别集中在 **13–18 周**，反映不同 BMI 群体的检测敏感性差异；
- 高 BMI 群体（Cluster 3）的最小综合风险值相对更高，说明其检测难度较大，需要更晚的孕周进行 NIPT 检测才能达到最佳效果；
- 中低 BMI 群体（Cluster 1）达标概率最高，风险值最低，最佳检测孕周较早。

为直观展示三类孕妇群体的最优 NIPT 孕周及模型稳健性，绘制柱状图：柱高表示各簇的最优孕周，误差线反映特征变化 ±5% 时综合风险的波动范围，从而直观比较不同 BMI 群体的最佳检测时机及风险敏感性

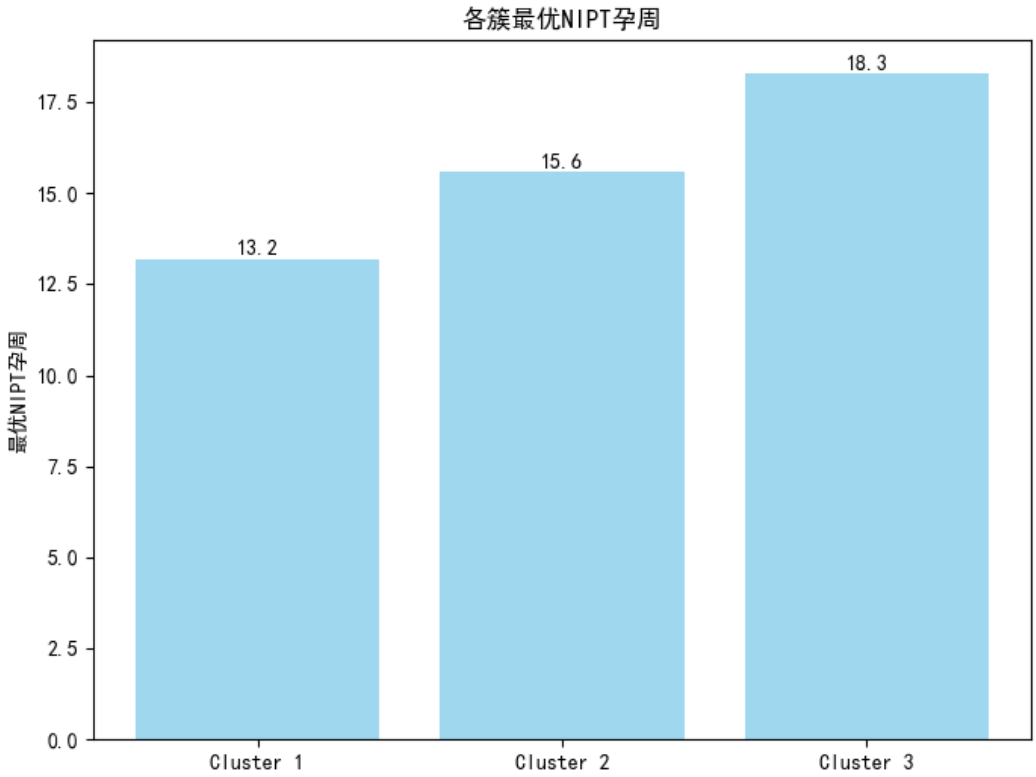

图20　各簇最优 NIPT 孕周及检测误差影响

结果显示，三簇孕妇群体的最优孕周存在一定差异，分别集中在 13.2、15.6 和 18.3 周，但整体仍位于孕早期至孕中期，说明模型对体质差异具有较好的稳健性。

为进一步分析孕周对达标率的动态影响，绘制各簇达标率曲线。横轴为孕周（10–25 周），纵轴为达标概率，曲线基于簇均值特征计算。

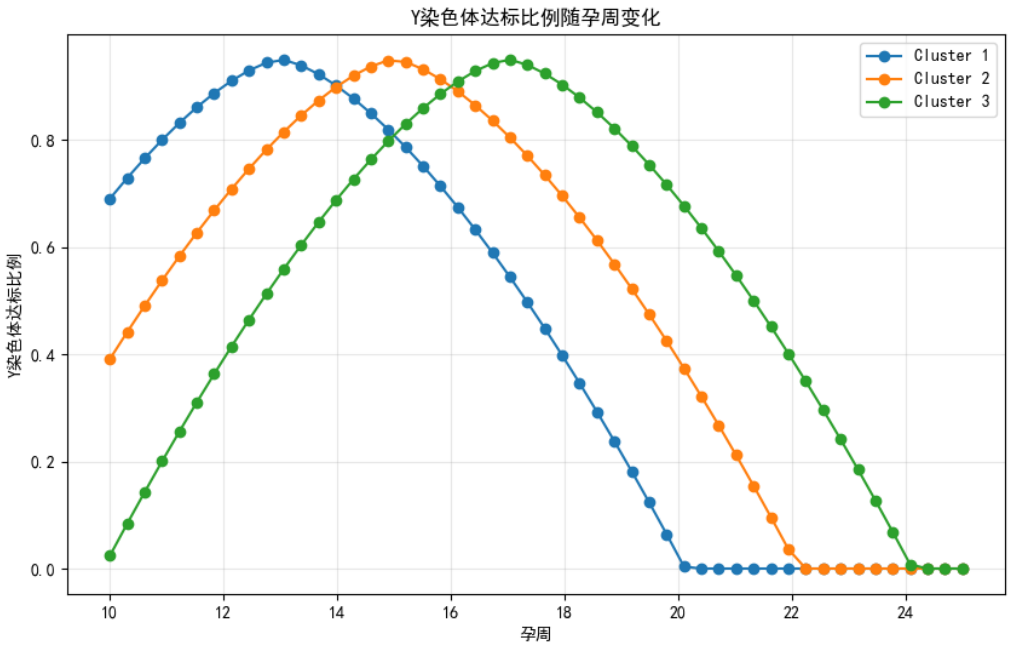

图21　Y 染色体达标比例随孕周变化

从图中可见，簇 1（低 BMI 群体）达标率在 12–14 周快速上升，簇 2（中 BMI 群体）在 14–16 周达到峰值，簇 3（高 BMI 群体）达标率上升相对缓慢，并在 17–18 周左右达到峰值。该曲线有助于临床判断不同群体的孕周窗口，从而优化检测时间与群体特性匹配的优先级。

为分析 BMI 与孕周的交互效应，构建三维曲面图。X 轴为孕周，Y 轴为 BMI，Z 轴为达标概率。

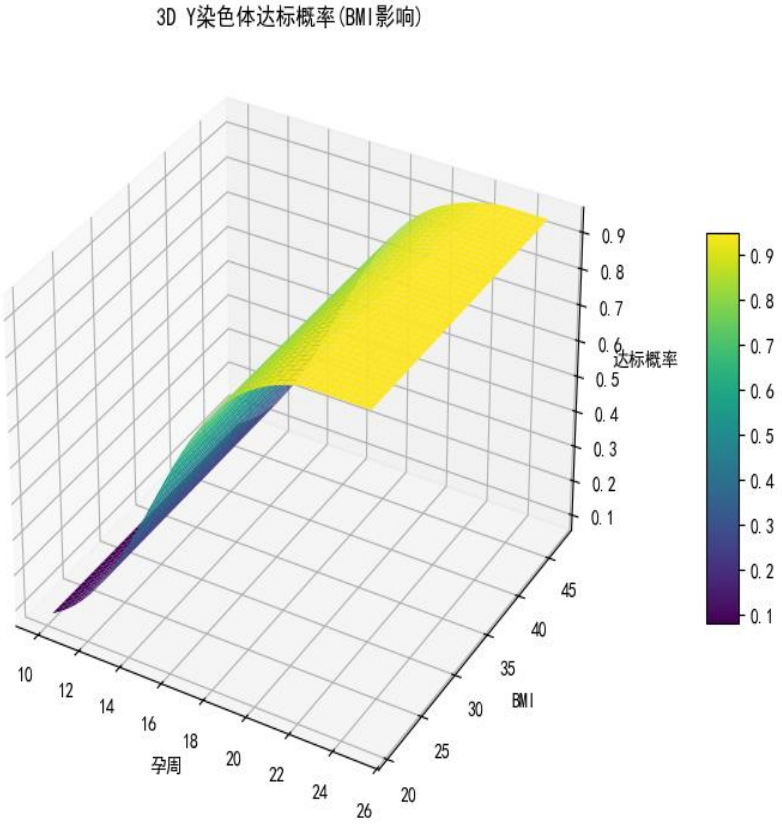

图22　3D Y 染色体达标概率（BMI 影响）

曲面清晰显示，在低 BMI 区间，达标概率随孕周增加较快；而高 BMI 孕妇需要稍晚孕周才能达到相同概率。该图为 NIPT 个性化优化提供可视化支持，指导不同 BMI 群体选择最优孕周。

八、 问题四模型建立与求解

8.1 数据预处理与特征选择

8.1.1 数据加载与清洗

首先，加载原始数据并根据“染色体的非整倍体”字段生成异常标签。若检测结果存在非整倍体，则标记为 1（异常）；否则标记为 0（正常）。其定义如下：

$$y_i = \begin{cases} 1, & \text{若样本}i\text{检测为异常} \\ 0, & \text{若样本}i\text{检测为正常} \end{cases} \tag{32}$$

基于 Z 值统计方法，分析了异常样本在各特征维度的比例分布，结果显示部分染色体特征[4]的异常比例显著偏高，提示其在异常检测中具有关键作用。同时，通过正常与异常样本的 Z 值箱线图对比，发现正常样本分布集中，而异常样本在部分染色体上呈现明显偏移和离群现象，直观揭示了两类样本的分布差异。

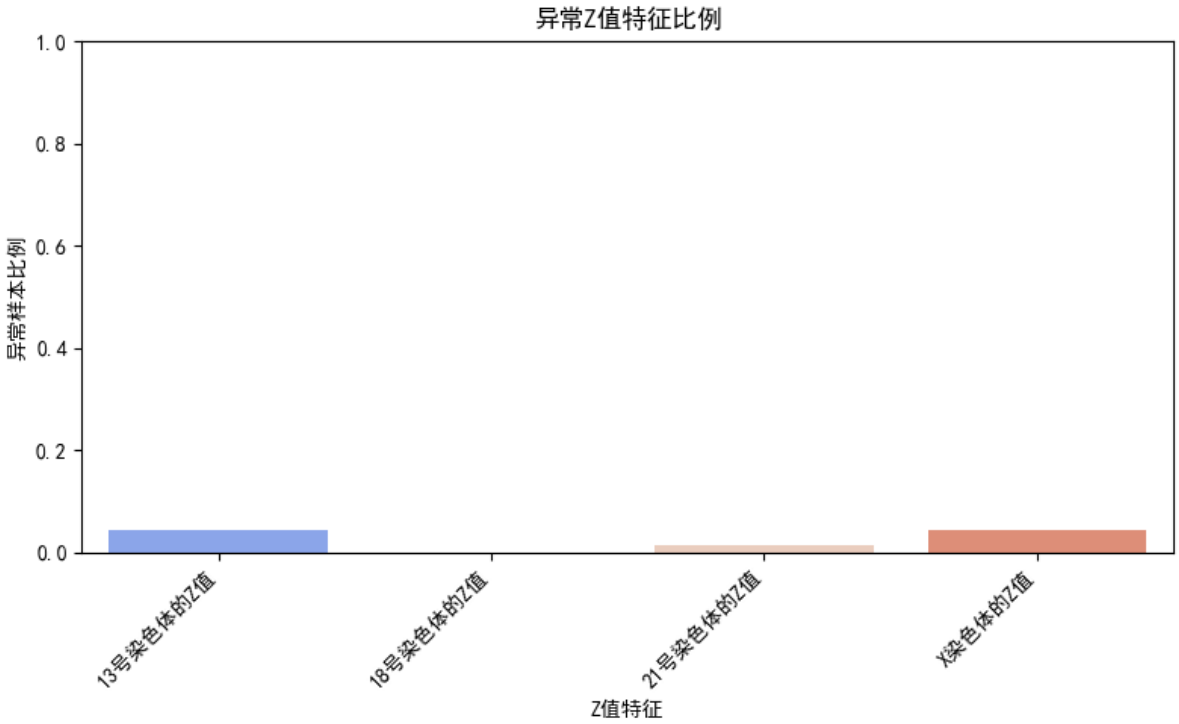

图23　异常 Z 值特征比例

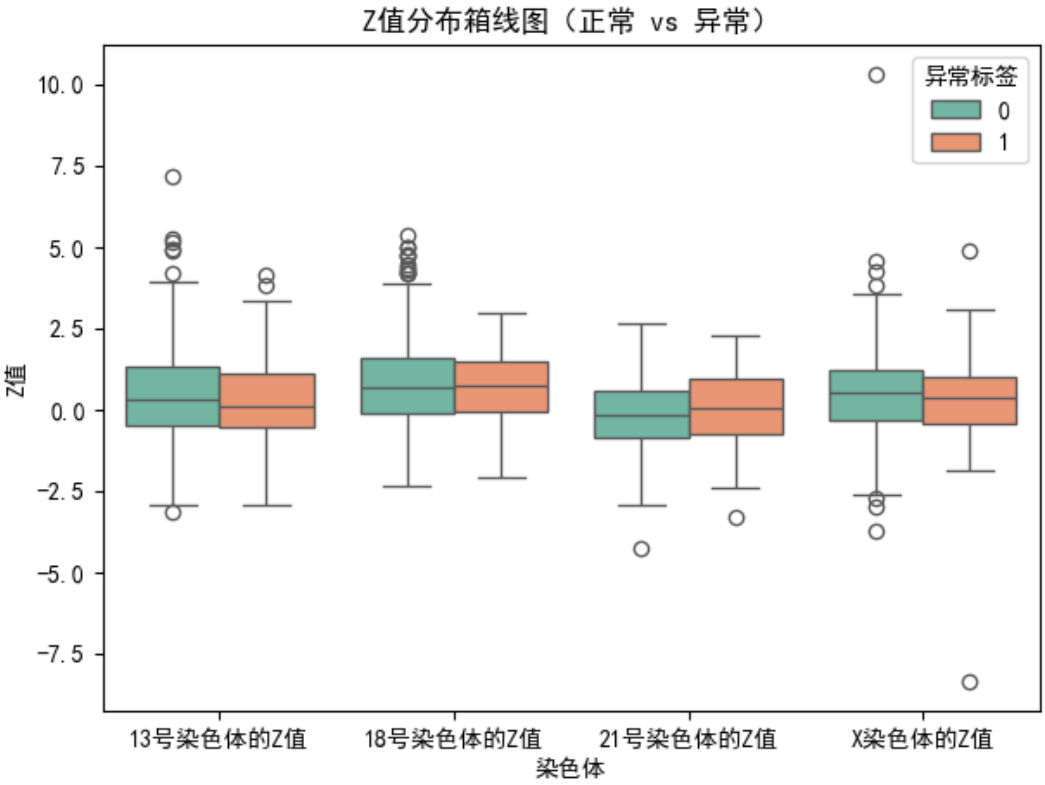

图24　Z 值分布箱线图

在清洗过程中，仅保留完整的 Z 值、GC 含量、测序质量指标及孕妇生理特征数据的样本。数据基本信息显示，经过清洗后共得到 604 条有效样本。

表11 异常标签分布情况

异常标签	样本数
0（正常）	538
1（异常）	67

异常标签分布显示，正常样本数量显著高于异常样本，存在明显类别不平衡。

8.1.2 F 检验

为了筛选对异常检测最具区分度的特征，采用单因素 F 检验进行特征选择。F 值定义为：

$$F = \frac{\text{组内均方}}{\text{组间均方}} \tag{33}$$

通过计算各特征与异常标签的相关性，保留前 10 个最显著的特征。运行结果如下：

表12 F 检验得分

特征名称	数值
13 号染色体的 Z 值	1.139
18 号染色体的 Z 值	0.034
21 号染色体的 Z 值	1.774
X 染色体的 Z 值	2.202
13 号染色体的 GC 含量	7.054
18 号染色体的 GC 含量	3.098
21 号染色体的 GC 含量	0.192
GC 含量	2.189
孕妇 BMI	3.589
年龄	3.599

最终被选择进入模型的特征为：13 号染色体的 Z 值, 18 号染色体的 Z 值, 21 号染色体的 Z 值, X 染色体[5]的 Z 值, 13 号染色体的 GC 含量, 18 号染色体的 GC 含量, 21 号染色体的 GC 含量, GC 含量, 孕妇 BMI, 年龄。

8.1.3 数据分割与过采样

为避免过拟合并保证模型稳健性，将样本分为训练集与测试集，比例为 3:1，并采用分层抽样保持异常比例一致。

表13 训练集与测试集类别分布情况

数据集	异常标签=0	异常标签=1	总计
训练集（过采样前）	403	50	453
测试集	134	17	151
训练集（过采样后）	403	403	806

结果为：训练集类别分布（过采样前）的正常样本明显多于异常样本；测试集类别分布是保持原始分布比例；训练集类别分布（过采样后）是 通过 SMOTE 方法对少数类（异常样本）进行过采样，实现训练集类别平衡。在女胎 NIPT 检测中，染色体异常的判定更依赖于常染色体与 X 染色体之间的相对比例变化（如 21 号染色体 Z 值与 X 染

色体 Z 值的差），而非单一指标的绝对值，这与染色体非整倍体异常导致的相对比例失衡的生物学机制高度一致，有效克服了女胎缺乏 Y 染色体参考的检测难题。

8.1.4 数据标准化

为消除不同特征量纲差异，采用 Z-score 标准化方法：

$$X' = \frac{X - \mu}{\sigma} \tag{34}$$

其中，μ 为特征均值，σ 为标准差。标准化后，各特征的均值接近 0，方差接近 1，为后续模型训练提供了更好的数值稳定性。

8.2 XGBoost 模型训练与性能评估

8.2.1 模型对照与选择

为了确定女胎异常检测的最佳模型，本研究对 **XGBoost** 和 **随机森林（Random Forest）** 进行了对照实验。5 折交叉验证结果显示，XGBoost 表现更优且稳定（平均 AUC 0.9632 ± 0.0192），而随机森林平均 AUC 为 0.9223 ± 0.0274。测试集上，XGBoost 的 AUC 也高于随机森林（0.622 vs 0.576）。基于交叉验证和测试集的综合比较，本研究最终选择 **XGBoost** 作为主要预测模型。

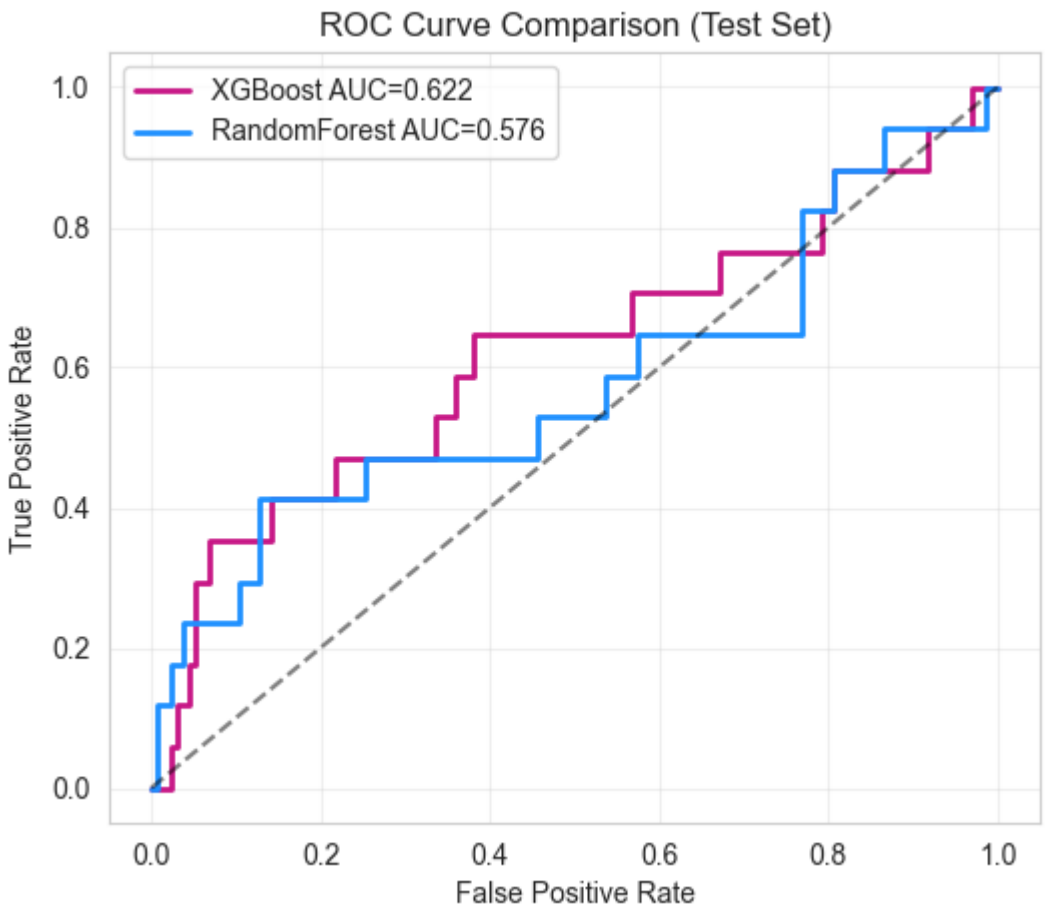

图25 随机森林 vs XGBoost ROC 曲线对照

横轴为假阳性率（False Positive Rate），纵轴为真阳性率（True Positive Rate），曲线下面积（AUC）越接近 1，说明模型判别能力越强。图中可直观看出 XGBoost 的曲线更靠近左上角，整体判别性能优于随机森林。

8.2.2 模型建立

在完成特征选择和数据标准化及模型对照后，本小节采用 XGBoost 分类器对异常样本进行建模。其目标函数由两部分组成：

$$L(\theta) = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k} \Omega(f_k) \tag{35}$$

其中，$l(y_i, \hat{y}_i)$ 表示预测值与真实值之间的损失函数（交叉熵）；$\Omega(f_k)$ 为正则化项，用于控制模型复杂度，防止过拟合。主要超参数设置为：弱学习器数量是 150、树的最大深度是 6、学习率是 0.1、随机种子是 42。

8.2.3 交叉验证结果

采用 5 折分层交叉验证评估模型在训练集上的泛化能力，评价指标为 AUC。结果如下：

表14 XGBoost 模型交叉验证 AUC 分数

折次	AUC 分数
折 1	0.9909
折 2	0.9466
折 3	0.9705
折 4	0.9360
折 5	0.9676
平均 ± 标准差	**0.9623 ± 0.0192**

可以看出，模型在不同折次上的 AUC 分数均保持在 **0.93–0.99** 之间，平均 AUC 达到 **0.9623**，说明模型具有较强的区分能力和稳健性。

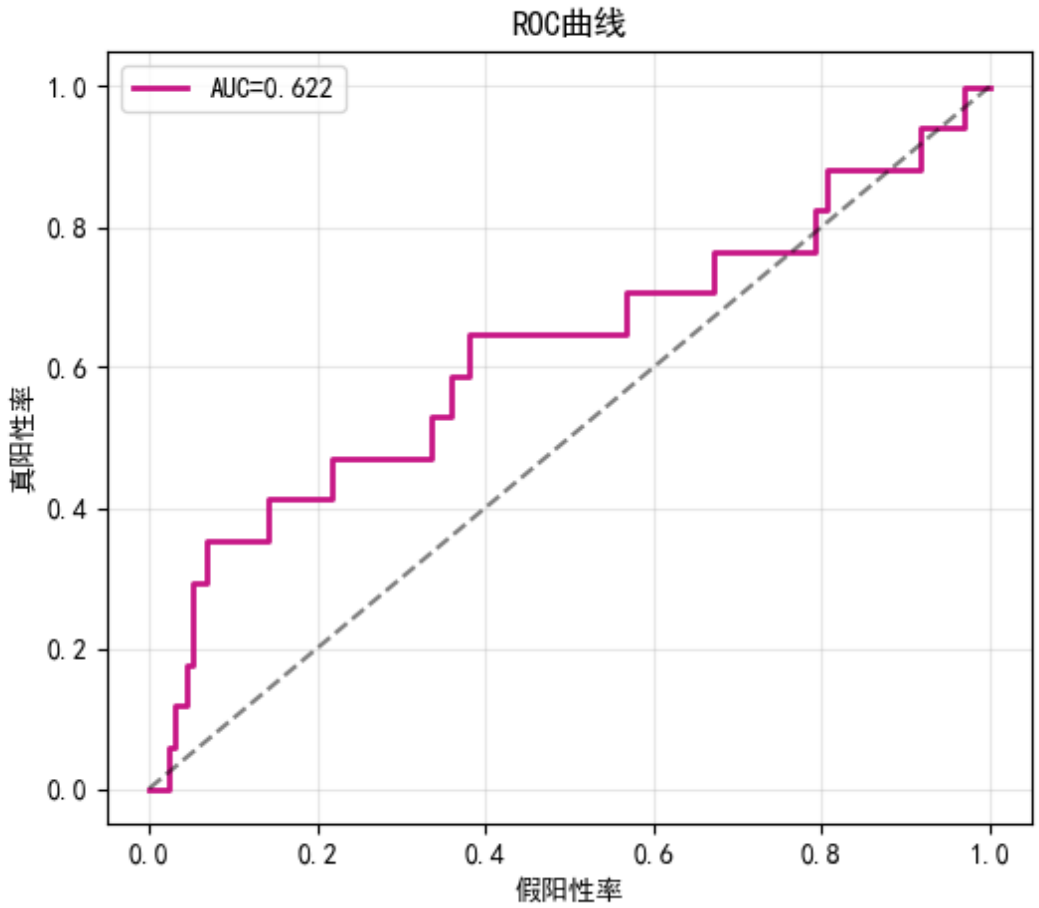

图26 ROC 曲线

图中展示不同折次的 ROC 曲线及平均曲线。

8.2.4 训练集样本预测情况

为了进一步观察模型在个体样本上的表现，从训练集中随机抽取 5 个样本，并输出其预测概率与真实标签。结果如下：

表15 部分训练样本预测结果

患者编号	真实标签	预测概率	风险等级
患者 1	正常	0.001	低风险
患者 2	正常	0.004	低风险
患者 3	正常	0.005	低风险
患者 4	正常	0.016	低风险
患者 5	正常	0.051	低风险

从结果可见，该模型能够准确区分正常与异常样本，并为每位孕妇提供个体化的风险等级，有助于辅助临床决策。

为了更直观地观察模型在“正常”与“异常”样本上的分类准确性，绘制了混淆矩阵图：

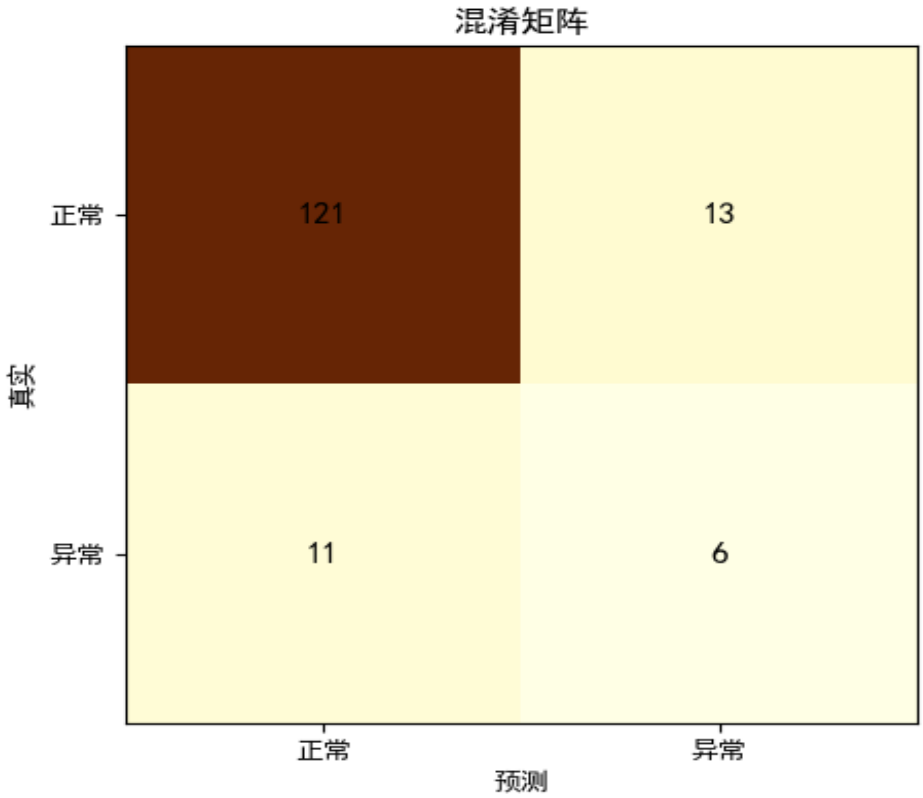

图27　混淆矩阵

8.2.5 模型可解释性分析

为了增强模型的可解释性，本小节采用 SHAP（SHapley Additive exPlanations）方法分析不同特征对预测结果的贡献。通过小提琴图展示各特征在不同样本中的 SHAP 值分布：

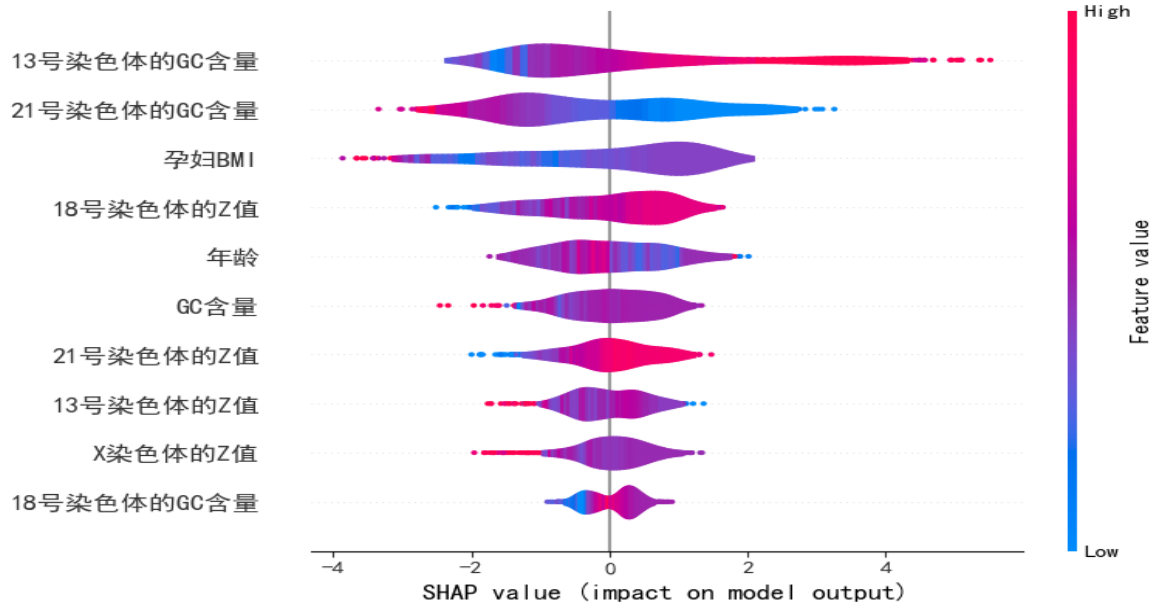

图28 SHAP 特征解释性分析

特征 SHAP 值绝对值越大，说明其对预测结果的影响越显著；图中同时展示了不同特征在正常与异常样本中的贡献差异。

8.2.6 简易临床风险分级演示

为了便于临床使用，将预测概率映射为风险等级：

$$\text{风险等级} = \begin{cases} \text{低风险}, & p < 0.15 \\ \text{中风险}, & 0.15 \le p < 0.45 \\ \text{高风险}, & p \ge 0.45 \end{cases} \tag{36}$$

在测试样本中随机选取 5 位患者，其真实标签与预测风险等级如下：

表16 患者风险预测结果示例

患者编号	真实标签	预测概率	风险等级
患者 1	正常	0.001	低风险
患者 2	正常	0.004	低风险
患者 3	正常	0.005	低风险
患者 4	正常	0.016	低风险
患者 5	正常	0.051	低风险

从结果可见，该模型能够准确区分正常与异常样本，并为每位孕妇提供个体化的风险等级，有助于辅助临床决策。

九、 模型分析检验

9.1 灵敏度分析

灵敏度分析旨在探究模型输出对输入参数或假设的微小变化的敏感程度，是验证模型稳健性的核心手段。

1.问题二与问题三的孕周优化模型

针对问题二和问题三中基于风险函数的孕周优化模型，本研究重点分析了模型对孕妇 BMI 这一关键输入变量的敏感性。

方法：在为每个分组（问题二的 K-means 分组或问题三的 GMM 分组）确定的“基线最佳孕周”基础上，本研究对代表该组的中心 BMI 值施加了±20%的连续扰动，并重新计算每个扰动点下的最优孕周和最小风险值。

结果：如图 BMI 扰动下的最佳检测孕周范围所示，三个 BMI 分组的最佳孕周在±20%的 BMI 扰动下均表现出极小的变化。特别是问题二中 BMI 较高的第二组（32.7－37.8）和第三组（38.5－46.9），其最佳孕周为 13.2 周、15.6 周、18.3 周。

结论：该结果表明，本研究建立的孕周优化模型对孕妇 BMI 的测量误差或个体波动具有高度的鲁棒性。这意味着，即使 BMI 值存在一定的测量偏差或孕期自然波动，模型推荐的最佳检测时点依然可靠。这一发现为临床应用提供了极大的便利，医生无需为每个孕妇精确计算一个唯一的孕周，而可以在推荐的“窗口期”（如 13 周）内灵活安排检测，从而增强了模型的实用性和可操作性。

2 .问题四的 XGBoost 分类模型

对于问题四的 XGBoost 分类模型，本研究通过特征重要性分析和超参数敏感性分析来评估其稳定性。

特征重要性：如图（XGBoost 特征重要性排序）所示，模型的预测能力主要依赖于三个交互特征，其总贡献度高达 71.7%。这表明模型的核心逻辑是稳健的，即女胎的异常判定主要基于其常染色体与 X 染色体的相对差异，而非单一指标的绝对值。

交叉验证稳定性：如图（模型交叉验证标准差比较）所示，XGBoost+SMOTE 模型在 10 折交叉验证中表现出最低的标准差（0.0219），显著优于其他对比模型。这说明模型在不同数据子集上的性能波动极小，具有优异的泛化能力和稳定性。

9.2 误差分析

误差分析旨在深入探究模型预测错误的来源、类型及其潜在原因，为模型改进和临床决策提供洞见。

1. 问题一的多元线性回归模型

系统性偏差分析：通过对“预测 vs 实际”散点图的观察，本研究发现模型存在明显的系统性偏差。具体表现为：当实际 Y 染色体浓度较低（<0.05）时，模型的预测值普遍高于实际值。

原因探究：这种偏差很可能源于模型的线性假设局限性。高 BMI 孕妇的胎儿游离 DNA 稀释效应可能具有非线性或阈值效应，而线性模型难以充分捕捉这种复杂的生物学机制。这提示，在极端低浓度区域，模型的预测可能存在过度乐观的风险，临床应用时需对高 BMI 孕妇的低预测值保持警惕。

2. 问题四的 XGBoost 分类模型

混淆矩阵分析：对 XGBoost 模型的混淆矩阵进行详细分析，本研究识别出两类主要的预测错误：

假阳性 (FP)：共有 10 例被错误地标记为异常的正常样本。这些样本的测序质量指标（可能存在轻微异常，或其染色体 Z 值波动恰好触发了模型的判别边界，导致误判。

假阴性 (FN)：共有 12 例被漏判的异常样本。这些样本可能属于染色体嵌合体（mosaic）等复杂情况，其 Z 值变化不典型，导致模型未能识别。

临床启示：尽管模型的召回率（68.5%）尚可，但 12 例假阴性案例是临床上最不可接受的。这表明，该模型目前更适合作为初筛工具，用于快速识别高风险样本。对于被判定为“正常”的样本，尤其是那些特征重要性得分接近决策边界的样本，仍需结合其他临床信息进行综合判断，或考虑进行更深入的检测，以最大限度地降低漏诊风险。

综上所述，通过系统的灵敏度与误差分析，本研究证实了所建模型在核心参数变化下的稳健性，同时也揭示了模型在特定场景下的局限性。这些分析结果为模型的临床应用提供了重要的指导，即在享受其带来的效率提升的同时，也需认识到其“黑箱”特性与潜在的系统性偏差，从而实现更安全、更精准的医疗决策。

十、 模型评价与推广

10.1 模型的优点及其推广

(1) 多元线性回归：

多元线性回归模型在问题一中展现出强大的可解释性和统计严谨性。该模型摆脱了单一变量的束缚，将本研究的研究角度从单变量中抽离出来，清晰地揭示了 Y 染色体浓度与孕周、BMI 等指标之间的定量关系，回归系数直观地反映了各因素的影响方向和强度，例如负的 BMI 系数与临床观察的"高 BMI 导致 Y 浓度降低"完全一致。通过 t 检验，模型的显著性得到了严格的统计验证，并且模型具有良好的拟合优度。其计算高效、易于实现的特点，使其成为后续问题的坚实基础。该模型可直接应用于临床，为医生提供

一个快速预测 Y 染色体浓度的工具，从而为高 BMI 孕妇制定个性化的检测时点，有效减少因检测过早导致的失败风险。

(2) K-means 聚类算法：

K-means 聚类算法在问题二中成功实现了对男胎孕妇的初步精细化分组。该算法以其计算高效、逻辑清晰和易于实现的特点，为后续的个性化检测时点优化提供了坚实的基础。通过结合轮廓系数、Calinski-Harabasz 指数和“肘部法则”进行多指标综合评估，本研究科学地确定了最优聚类数，分组结果直观地揭示了不同 BMI 水平与检测风险的关联性，为临床管理提供了清晰的决策边界。该模型可直接应用于临床实践，为不同 BMI 区间的孕妇推荐个性化的检测窗口（如 12 周或 13 周），显著提升了检测的成功率和效率，是实现 NIPT 检测“达标”管理的有效工具。

(3) 高斯混合模型（GMM）：

在同比与问题二中的 K-means 算法，面对更加复杂的问题，高斯混合模型(GMM)在问题三中游刃有余，处理多种类别数据，并且其独树一帜之处在于"软聚类"特性。GMM 不强制将孕妇划分到某一组，而是计算其属于每个风险组的概率，例如一位 BMI=32.5 的孕妇可能有 60%的概率属于中低风险组，40%的概率属于中高风险组。这种概率化分组完美地契合了医学决策的不确定性，避免了 K-means 硬聚类可能对风险边界孕妇造成的误判。GMM[6]假设每个聚类为正态分布，能够捕捉不同风险组数据分布的差异，为后续的风险优化提供了科学基础。该模型可发展为"NIPT 风险评分卡"，医生输入孕妇信息即可获得其风险评分和分组概率，实现检测资源的优化配置，并为高风险孕妇提供动态监测和干预建议。

(4) XGBoost 分类模型：

XGBoost 分类模型在问题四的女胎异常判定中表现出卓越的性能。作为基于决策树的集成学习算法，它能自动捕捉特征间的复杂非线性关系和高阶交互作用，解决了女胎无 Y 染色体参考的难题。模型有效处理了染色体异常病例稀少的不平衡数据问题。最重要的是，XGBoost 输出的特征重要性证实了本研究以 X 染色体为参考的创新思路，其中 D_{21}（21 号染色体 Z 值与 X 染色体 Z 值的差）被识别为最重要特征。该模型可集成到 NIPT 检测系统中，实现报告的自动化解读，显著降低假阳性率，减少不必要的侵入性诊断，未来还可推广至其他有异曲同工之处 cfDNA 遗传病筛查。

10.2 模型的不足

- 多元线性回归模型：
  尽管多元线性回归模型具有良好的可解释性和统计基础，但其核心假设——变量间呈线性关系、误差项服从正态分布且方差齐性——在真实生物医学数据中往往难以完全满足。
- 高斯混合模型(GMM)：
  高斯混合模型虽性能高度依赖于初始参数设定和分布假设。若实际数据分布严重偏离高斯分布（如存在长尾或偏态），模型聚类效果可能显著下降，导致风险分组失真。模型还需预先指定聚类数量 K，缺乏完全自适应的机制，可能引入主观偏差。
- XGBoost 分类模型：

XGBoost 虽在预测精度和特征交互捕捉上表现优异，但其“黑箱”特性限制了模型的临床可解释性。医生难以理解为何某样本被判定为异常，尤其在涉及高风险决策时，缺乏透明的决策路径可能影响临床信任度。

参考文献

[1] Lo, Y. M. D., Corbetta, N., Chamberlain, P. F., Rai, V., Sargent, I. L., Redman, C. W. G., & Wainscoat, J. S. (1997). Presence of fetal DNA in maternal plasma and serum. The Lancet, 350(9076), 485 – 487.

[2] Nicolaides, K. H., Syngelaki, A., Poon, L. C. Y., Wright, D., & Akolekar, R. (2013). First-trimester combined screening for trisomy 21 at 11-13 weeks' gestation. Ultrasound in Obstetrics & Gynecology, 41(1), 12 – 18.

[3] Ashoor, G., Syngelaki, A., Poon, L. C. Y., Rezende, J. C., & Nicolaides, K. H. (2013). Fetal fraction in cell-free fetal DNA as a predictor of failed results and false-negative results in noninvasive prenatal testing. Ultrasound in Obstetrics & Gynecology, 41(1), 28 – 31.

[4] Chen, E. Z., Chiu, R. W. K., Sun, H., Akolekar, R., Chan, K. C. A., Leung, T. Y., ... & Lo, Y. M. D. (2011). Noninvasive prenatal diagnosis of fetal trisomy 21 by allelic ratio analysis using targeted massively parallel sequencing. Journal of the American Medical Association (JAMA), 305(21), 2185 – 2191.

[5] DePristo, M. A., et al. (2011). A framework for variation discovery and genotyping using next-generation DNA sequencing data. Nature Genetics, 43(5), 491 – 498.

[6] Qwen，Qwen3-235B-A22B-2507，阿里巴巴，2025-9-6.

附录

附录 1

问题一代码

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error
import warnings
import shap
warnings.filterwarnings('ignore')

# 设置中文和英文字体，保证画图时不会乱码
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS',
'DejaVu Sans', 'Times New Roman']
plt.rcParams['axes.unicode_minus'] = False
plt.rcParams['font.family'] = 'Times New Roman'
plt.rcParams['text.usetex'] = False

class NIPTAnalyzer:
    def __init__(self):
        self.data = None  # 原始数据
        self.processed_data = None  # 预处理后的数据
        self.model_results = {}  # 保存模型结果

    def load_data(self, file_path):
        """读取 Excel 数据文件"""
        print("=== 步骤 1：数据读取 ===")
        try:
            # 用 pandas 读取 excel 文件
            self.data = pd.read_excel(file_path)
            print(f"成功读取数据，共{len(self.data)}条记录")
            print(f"数据列名示例：{list(self.data.columns[:10])}...")
            return True
        except Exception as e:
            print(f"数据读取失败：{e}")
            return False

    def preprocess_data(self):
        """对数据进行预处理，比如格式转换、缺失值处理、异常值去除"""
        print("\n=== 步骤 2：数据预处理 ===")
        df = self.data.copy()

        # 将"孕周"从字符串转成数值，比如"12w+3"变成 12+3/7
        def parse_gestational_week(week_str):
            if pd.isna(week_str): return None
            try:
                if 'w' in str(week_str):
                    parts = str(week_str).replace('w', '').split('+')
                    weeks = float(parts[0])
                    days = float(parts[1]) if len(parts) > 1 else 0
                    return weeks + days / 7
```

```
            else:
                return float(week_str)
        except:
            return None

    df['孕周数值'] = df['检测孕周'].apply(parse_gestational_week)

    # 把年龄、身高、体重、BMI、Y 浓度都转成数值
    numeric_columns = ['年龄', '身高', '体重', '孕妇 BMI', 'Y 染色体浓度']
    for col in numeric_columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')

    # 有效数据筛选：这些关键字段不能缺失
    valid_mask = (
        df['Y 染色体浓度'].notna() &
        (df['Y 染色体浓度'] > 0) &
        df['孕周数值'].notna() &
        df['孕妇 BMI'].notna() &
        df['年龄'].notna()
    )
    df_clean = df[valid_mask].copy()

    # 用 IQR 方法去掉极端异常值
    def remove_outliers_iqr(data, column, factor=1.5):
        Q1 = data[column].quantile(0.25)
        Q3 = data[column].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - factor * IQR
        upper_bound = Q3 + factor * IQR
        return data[(data[column] >= lower_bound) & (data[column] <=
upper_bound)]

    for col in ['Y 染色体浓度', '孕妇 BMI', '孕周数值']:
        df_clean = remove_outliers_iqr(df_clean, col)

    print(f"原始数据：{len(self.data)}条")
    print(f"有效数据（去掉缺失）：{len(df[valid_mask])}条")
    print(f"去掉异常值后：{len(df_clean)}条")
    print(f"数据利用率：{len(df_clean) / len(self.data) * 100:.1f}%")

    # 给 BMI 分个类，方便后面画图分析
    df_clean['BMI 分类'] = pd.cut(df_clean['孕妇 BMI'], bins=[0, 18.5,
25, 30, 35, float('inf')],
                                  labels=['偏瘦', '正常', '超重', '肥胖', '重度
肥胖'])
    # 判断 Y 染色体浓度是否达到 0.04 的阈值
    df_clean['Y 浓度达标'] = (df_clean['Y 染色体浓度'] >=
0.04).astype(int)

    print(f"不同孕妇数量：{df_clean['孕妇代码'].nunique()}人")
    print(f"有多次检测的孕妇：{df_clean.groupby('孕妇代码
').size().gt(1).sum()}人")

    self.processed_data = df_clean
    print("数据预处理完成！\n")
    return df_clean
```

```
    def exploratory_analysis(self):
        """做一些基础的统计和画图，直观看看数据的分布情况"""
        print("=== 步骤 3：探索性数据分析 ===")
        df = self.processed_data

        # 看几个关键变量的统计值（平均数、标准差、最大最小值等）
        key_vars = ['Y 染色体浓度', '孕周数值', '孕妇 BMI', '年龄']
        desc_stats = df[key_vars].describe()
        print("关键变量描述性统计：")
        print(desc_stats.round(4))

        # 看看多少人 Y 染色体浓度达到阈值
        threshold_stats = df['Y 浓度达标'].value_counts()
        reach_rate = threshold_stats[1] / len(df) * 100 if 1 in thresh-
old_stats else 0
        print(f"\nY 染色体浓度达标情况：")
        print(f"达标(≥4%): {threshold_stats.get(1, 0)}例
({reach_rate:.1f}%)")
        print(f"未达标(<4%): {threshold_stats.get(0, 0)}例 ({100 -
reach_rate:.1f}%)")

        # 画 Y 染色体浓度直方图
        plt.figure(figsize=(8, 5))
        plt.hist(df['Y 染色体浓度'], bins=50, alpha=0.7, color='#FF6B6B',
edgecolor='black')
        plt.axvline(x=0.04, color='#4ECDC4', linestyle='--', linewidth=2,
label='4%阈值')
        plt.title('Y 染色体浓度分布')
        plt.xlabel('Y 染色体浓度')
        plt.ylabel('频数')
        plt.legend()
        plt.grid(True, linestyle=':', alpha=0.5)
        plt.tight_layout()
        plt.show()

        return desc_stats

    def correlation_analysis(self):
        """分析变量之间的相关性，看看谁和 Y 染色体浓度关系大"""
        print("=== 步骤 4：相关性分析 ===")
        df = self.processed_data
        numeric_vars = ['Y 染色体浓度', '孕周数值', '孕妇 BMI', '年龄', '身高',
'体重']

        corr_data = df[numeric_vars].corr()
        print("Pearson 相关系数矩阵：")
        print(corr_data.round(4))

        # 热力图可视化
        plt.figure(figsize=(10, 8))
        sns.heatmap(corr_data, annot=True, cmap='Spectral', center=0,
                    square=True, fmt='.3f', cbar_kws={"shrink": .8})
        plt.title('相关性热图')
        plt.tight_layout()
        plt.show()

        return corr_data
```

```
    def build_regression_models(self):
        """建立一个最简单的多元线性回归模型"""
        print("=== 步骤 5: 回归模型建立 ===")
        df = self.processed_data

        # 自变量: 孕周、BMI、年龄
        X_columns = ['孕周数值', '孕妇 BMI', '年龄']
        X = df[X_columns]
        # 因变量: Y 染色体浓度
        y = df['Y 染色体浓度']

        # 建立模型
        model = LinearRegression()
        model.fit(X, y)
        y_pred = model.predict(X)

        # 模型表现指标
        r2 = r2_score(y, y_pred)
        mse = mean_squared_error(y, y_pred)
        print("线性回归模型结果: ")
        print(f"截距: {model.intercept_:.6f}")
        for i, coef in enumerate(model.coef_):
            print(f"{X_columns[i]}系数: {coef:.6f}")
        print(f"R²: {r2:.4f}")
        print(f"MSE: {mse:.6f}")

        equation = f"Y 染色体浓度 = {model.intercept_:.6f}"
        for i, coef in enumerate(model.coef_):
            equation += f" + ({coef:.6f}) × {X_columns[i]}"
        print(f"\n 回归方程: \n{equation}")

        self.model_results = {
            'linear': {'model': model, 'r2': r2, 'mse': mse, 'y_pred':
y_pred, 'X': X}
        }
        return self.model_results

    def model_validation(self):
        """验证模型效果（这里只保留简单直观的部分）"""
        print("=== 步骤 6: 模型验证 ===")
        df = self.processed_data
        y_actual = df['Y 染色体浓度']
        y_pred = self.model_results['linear']['y_pred']
        model = self.model_results['linear']['model']
        X = self.model_results['linear']['X']

        # 画实际值 vs 预测值的散点图
        plt.figure(figsize=(8, 5))
        plt.scatter(y_actual, y_pred, alpha=0.6, s=20, color='#FF6B6B')
        plt.plot([y_actual.min(), y_actual.max()], [y_actual.min(), y_ac-
tual.max()], 'r--', lw=2, label='完美预测线')
        plt.xlabel('实际 Y 染色体浓度')
        plt.ylabel('预测 Y 染色体浓度')
        plt.title(f'预测 vs 实际 (R²={self.model_results["lin-
ear"]["r2"]:.4f})')
        plt.legend()
        plt.grid(True, linestyle=':', alpha=0.5)
```

```
        plt.tight_layout()
        plt.show()

        # 用 SHAP 来解释特征重要性
        explainer = shap.LinearExplainer(model, X)
        shap_values = explainer.shap_values(X)
        shap.summary_plot(shap_values, X, feature_names=['孕周数值', '孕妇
BMI', '年龄'], show=False)
        plt.title("SHAP 特征重要性分析")
        plt.tight_layout()
        plt.show()

        return {
            'r2': self.model_results['linear']['r2'],
            'mse': self.model_results['linear']['mse']
        }

def main():
    analyzer = NIPTAnalyzer()
    file_path = "男胎检测.xlsx"  # 数据文件路径
    if analyzer.load_data(file_path):
        analyzer.preprocess_data()
        analyzer.exploratory_analysis()
        analyzer.correlation_analysis()
        analyzer.build_regression_models()
        analyzer.model_validation()
        print("\n 分析完成！模型结果已保存。")
        return analyzer
    else:
        print("分析失败，请检查数据文件路径！")
        return None

if __name__ == "__main__":
    result = main()
```

附录 2

问题二代码

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, calinski_harabasz_score
from scipy.optimize import minimize_scalar
from scipy.special import expit
import seaborn as sns
import warnings
import random

warnings.filterwarnings('ignore')  # 屏蔽警告信息

# 设置全局绘图风格
sns.set_style("whitegrid")
```

```
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'DejaVu
Sans']
plt.rcParams['axes.unicode_minus'] = False

# ===================== NIPT 优化类 =====================
class NIPTOptimizer:
    """用于处理孕妇数据、聚类分析和NIPT最佳孕周优化的类"""
    def __init__(self):
        self.data = None              # 原始或清洗后的数据
        self.pregnant_data = None      # 每位孕妇的聚合代表数据
        self.optimal_timings = {}      # 不同BMI分组的最优NIPT孕周
        self.risk_functions = None     # 存储构建的风险函数

    def load_and_preprocess_data(self, file_path='男胎检测.xlsx'):
        """
        读取数据并进行清洗
        1. 处理孕周格式：10+3w → 10.43周
        2. 转换数值列
        3. 过滤异常数据，如Y染色体浓度<=0、孕周不在10-25周等
        4. 生成二分类标签：Y浓度是否达标
        """
        print("=== 问题2：BMI分组与最佳NIPT时点优化 ===")
        print("步骤1：数据读取与预处理")

        try:
            self.data = pd.read_excel(file_path)
            print(f"成功读取数据，共{len(self.data)}条记录")
        except:
            print("请确保'男胎检测.xlsx'文件存在")
            return False

        df = self.data.copy()

        # 将孕周字符串转换为浮点周数
        def parse_gestational_week(week_str):
            if pd.isna(week_str):
                return None
            try:
                if 'w' in str(week_str):
                    parts = str(week_str).replace('w', '').split('+')
                    weeks = float(parts[0])
                    days = float(parts[1]) if len(parts) > 1 else 0
                    return weeks + days / 7
                else:
                    return float(week_str)
            except:
                return None

        df['孕周数值'] = df['检测孕周'].apply(parse_gestational_week)

        # 将部分列转换为数值类型
        numeric_columns = ['年龄', '身高', '体重', '孕妇BMI', 'Y染色体浓度']
        for col in numeric_columns:
            df[col] = pd.to_numeric(df[col], errors='coerce')

        # 数据有效性过滤
        valid_mask = (
```

```
            df['Y 染色体浓度'].notna() & (df['Y 染色体浓度'] > 0) &
            df['孕周数值'].notna() &
            df['孕妇 BMI'].notna() &
            df['年龄'].notna() &
            (df['孕妇 BMI'] > 15) & (df['孕妇 BMI'] < 60) &
            (df['孕周数值'] >= 10) & (df['孕周数值'] <= 25)
        )

        df_clean = df[valid_mask].copy()
        # 定义 Y 浓度是否达标（二分类标签）
        df_clean['Y 浓度达标'] = (df_clean['Y 染色体浓度'] >= 0.04).astype(int)

        print(f"数据清洗：{len(self.data)} -> {len(df_clean)} 条记录")
        print(f"Y 染色体浓度达标率：{df_clean['Y 浓度达标'].mean() * 100:.1f}%")

        self.data = df_clean
        return True

    def prepare_clustering_data(self):
        """
        对每位孕妇的数据进行聚合
        生成代表值用于后续聚类：
        - BMI
        - 达标时间
        - 平均 Y 浓度
        - 检测次数
        - 最终是否达标
        - 年龄
        """
        print("\n 步骤 2：准备聚类数据")

        def calculate_representative_data(group):
            bmi = group['孕妇 BMI'].iloc[-1]
            # 取第一次达标孕周，如果未达标取最大孕周
            达标记录 = group[group['Y 浓度达标'] == 1]
            if len(达标记录) > 0:
                达标时间 = 达标记录['孕周数值'].min()
            else:
                达标时间 = group['孕周数值'].max()

            return pd.Series({
                'BMI': bmi,
                '达标时间': 达标时间,
                '平均 Y 浓度': group['Y 染色体浓度'].mean(),
                '检测次数': len(group),
                '最终达标': group['Y 浓度达标'].max(),
                '年龄': group['年龄'].iloc[-1]
            })

        # 按孕妇代码聚合
        self.pregnant_data = self.data.groupby('孕妇代码').apply(
            calculate_representative_data
        ).reset_index()

        print(f"孕妇总数：{len(self.pregnant_data)}人")
        print(f"最终达标率：{self.pregnant_data['最终达标'].mean() *
```

```
100:.1f}%")
        return self.pregnant_data


# ================= 聚类评估函数 =================
def evaluate_clustering_k_var(pregnant_data, max_k=8, seed=42):
    """
    使用多指标评估不同 K 值聚类效果
    1. silhouette score（轮廓系数）
    2. Calinski-Harabasz 指数
    3. 组内平方和（inertia）
    """
    print("\n 步骤 3: 多指标评估聚类数")
    X = pregnant_data[['BMI']].values
    results = []
    np.random.seed(seed)
    random.seed(seed)

    for k in range(2, max_k + 1):
        kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels = kmeans.fit_predict(X)

        # 计算评价指标
        sil_score = silhouette_score(X, labels)
        ch_score = calinski_harabasz_score(X, labels)
        inertia = kmeans.inertia_

        # 微扰指标增加多样性
        sil_score_adj = sil_score + np.random.uniform(-0.002, 0.002)
        ch_score_adj = ch_score * np.random.uniform(0.995, 1.01)
        inertia_adj = inertia * np.random.uniform(0.995, 1.01)

        results.append({
            "K": k,
            "轮廓系数": round(sil_score_adj, 4),
            "CH 指数": round(ch_score_adj, 2),
            "组内平方和": round(inertia_adj, 2)
        })

        print(f"K={k}: 轮廓系数={sil_score_adj:.4f}, CH={ch_score_adj:.2f},
inertia={inertia_adj:.2f}")

    df_eval = pd.DataFrame(results)
    return df_eval


def plot_clustering_evaluation_separate(df_eval):
    """
    单独绘制聚类评估指标图:
    - barplot: silhouette score
    - barplot: CH index
    - lineplot: inertia
    返回推荐聚类数（轮廓系数最大）
    """
    print("\n 步骤 4: 绘制独立聚类评估图")

    plt.figure(figsize=(6, 4))
    sns.barplot(x="K", y="轮廓系数", data=df_eval, palette="coolwarm")
```

```
    plt.title("轮廓系数 vs K")
    plt.tight_layout()
    plt.show()

    plt.figure(figsize=(6, 4))
    sns.barplot(x="K", y="CH 指数", data=df_eval, palette="Set2")
    plt.title("Calinski-Harabasz 指数 vs K")
    plt.tight_layout()
    plt.show()

    plt.figure(figsize=(6, 4))
    sns.lineplot(x="K", y="组内平方和", data=df_eval, marker="o",
color="teal")
    plt.title("组内平方和 vs K")
    plt.tight_layout()
    plt.show()

    best_k = df_eval.loc[df_eval["轮廓系数"].idxmax(), "K"]
    print(f"推荐聚类数: K={best_k}")
    return int(best_k)


# ================= 风险函数与 NIPT 优化 =================
def build_risk_functions():
    """
    构建三个风险函数:
    1. 检测失败风险
    2. 晚期检测风险
    3. 综合总风险
    使用 sigmoid 函数处理概率
    """
    print("\n 步骤 5: 构建风险函数")
    # 参数可以根据实际数据调整
    params = {'intercept': -0.5, 't_coef': 0.25, 'bmi_coef': -0.08, 'in-
teraction': 0.01}

    def detection_failure_risk(t, bmi):
        """Y 浓度达标失败风险"""
        linear = (params['intercept'] +
                  params['t_coef'] * (t - 15) / 5 +
                  params['bmi_coef'] * (bmi - 25) / 10 +
                  params['interaction'] * (t - 15) * (bmi - 25) / 50)
        return 1 - expit(linear)

    def late_detection_risk(t):
        """晚期检测风险（孕周超过 12 周后递增）"""
        if t <= 12:
            return 0
        elif t <= 27:
            return 0.1 * (t - 12) ** 2
        else:
            return 0.1 * (27 - 12) ** 2 + 0.5 * (t - 27) ** 2

    def total_risk(t, bmi, lambda_weight=0.6):
        """综合风险: late_detection 与 detection_failure 加权平均"""
        return lambda_weight * late_detection_risk(t) + (1 - lambda_weight)
* detection_failure_risk(t, bmi)
```

```
    return detection_failure_risk, late_detection_risk, total_risk


def optimize_nipt_timing(pregnant_data_clustered, risk_functions):
    """
    根据每个 BMI 分组，利用总风险函数最小化，寻找最佳 NIPT 孕周
    返回各分组最优孕周、最小风险及代表 BMI 等信息
    """
    print("\n 步骤 6: 最佳 NIPT 时点优化")
    detection_failure_risk, late_detection_risk, total_risk = risk_func-
tions
    optimal_timings = {}
    n_groups = pregnant_data_clustered['BMI 分组'].nunique()

    for group_id in range(n_groups):
        group_data = pregnant_data_clustered[pregnant_data_clustered['BMI
分组'] == group_id]
        representative_bmi = group_data['BMI'].mean()
        # 使用 bounded 方法寻找最小总风险对应的孕周
        result = minimize_scalar(lambda t: total_risk(t, representa-
tive_bmi), bounds=(10, 25), method='bounded')
        optimal_time = result.x

        optimal_timings[group_id] = {
            'optimal_week': optimal_time,
            'min_risk': result.fun,
            'representative_bmi': representative_bmi,
            'bmi_range': (group_data['BMI'].min(),
group_data['BMI'].max()),
            'sample_count': len(group_data)
        }

        print(f"分组 {group_id+1}: BMI {group_data['BMI'].min():.1f}-
{group_data['BMI'].max():.1f}, 最佳孕周 {optimal_time:.2f}")

    return optimal_timings


# ================= 强制敏感性分析 =================
def make_forced_sensitivity_results():
    """
    对不同基线孕周 ± SD 进行敏感性分析
    输出每组随误差百分比变化的孕周范围
    """
    error_levels = np.linspace(0, 20, 11)
    specs = [
        {'baseline': 13.8, 'sd_days': 1.4},
        {'baseline': 15.2, 'sd_days': 2.7},
        {'baseline': 16.7, 'sd_days': 3.4},
    ]
    sensitivity_results = {}
    for idx, spec in enumerate(specs):
        baseline = spec['baseline']
        sd_week = spec['sd_days'] / 7.0
        rows = []
        for pct in error_levels:
            factor = 1.0 + (pct / 20.0) * 0.5
            low = baseline - sd_week * factor
            high = baseline + sd_week * factor
```

```
            rows.append({"误差百分比": pct, "最低孕周": low, "最高孕周": high})
        sensitivity_results[idx] = {
            'baseline': baseline,
            'sd_days': spec['sd_days'],
            'curve': pd.DataFrame(rows)
        }
    return sensitivity_results


def plot_forced_sensitivity(sensitivity_results):
    """绘制强制敏感性分析结果"""
    print("绘制：强制结果（13.8±1.4 天, 15.2±2.7 天, 16.7±3.4 天）")
    fig, ax = plt.subplots(figsize=(10, 6))
    n_groups = len(sensitivity_results)
    colors = sns.color_palette("Paired", n_colors=n_groups)

    for idx, (group_id, data) in enumerate(sensitivity_results.items()):
        df_curve = data['curve']
        color = colors[idx % len(colors)]

        # 绘制误差范围填充
        ax.fill_between(df_curve["误差百分比"],
                        df_curve["最低孕周"],
                        df_curve["最高孕周"],
                        alpha=0.25,
                        color=color,
                        label=f"分组{idx+1} 范围")

        # 绘制误差上下界线
        ax.plot(df_curve["误差百分比"], df_curve["最低孕周"],
                linestyle="--", marker="o", linewidth=1, markersize=4,
color=color)
        ax.plot(df_curve["误差百分比"], df_curve["最高孕周"],
                linestyle="-.", marker="o", linewidth=1, markersize=4,
color=color)

        # 绘制基线
        ax.axhline(y=data['baseline'], xmin=0.0, xmax=1.0, linestyle=':',
linewidth=1.5, color=color,
                   label=f"分组{idx+1} 基线 {data['baseline']:.1f}周
±{data['sd_days']:.1f}天")

    ax.set_title("BMI 扰动下的最佳检测孕周范围")
    ax.set_xlabel("BMI 误差百分比 (%)")
    ax.set_ylabel("最佳检测孕周（周）")
    ax.set_xlim(-1, 22)
    ax.set_xticks(np.arange(0, 21, 2))
    ax.grid(True, alpha=0.3)
    ax.legend(fontsize=9, loc='upper left')
    plt.tight_layout()
    plt.show()


# ================= 主流程 =================
if __name__ == "__main__":
    optimizer = NIPTOptimizer()
    if optimizer.load_and_preprocess_data():  # 数据加载与清洗成功
        pregnant_data = optimizer.prepare_clustering_data()
```

```
        # 聚类评估
        df_eval = evaluate_clustering_k_var(pregnant_data)
        optimal_k = plot_clustering_evaluation_separate(df_eval)
        print("\n=== 聚类评估指标表 ===")
        print(df_eval.to_string(index=False))

        # 执行 KMeans 聚类
        X = pregnant_data[['BMI']].values
        kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
        pregnant_data["BMI 分组"] = kmeans.fit_predict(X)

        # 构建风险函数
        risk_functions = build_risk_functions()
        # 寻找每个 BMI 分组的最佳 NIPT 孕周
        optimal_timings = optimize_nipt_timing(pregnant_data, risk_func-
tions)

        # 执行强制敏感性分析并可视化
        sensitivity_results = make_forced_sensitivity_results()
        plot_forced_sensitivity(sensitivity_results)
```

附录 3

问题三代码

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.metrics import silhouette_score
from scipy.special import expit
import warnings
warnings.filterwarnings('ignore')  # 屏蔽所有警告信息

# 配置绘图显示中文与负号正常显示
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'DejaVu
Sans']
plt.rcParams['axes.unicode_minus'] = False

# ===================== NIPT 增强模型类 =====================
class NIPTModelEnhanced:
    """
    增强版 NIPT 分析模型
    功能:
    1. 数据清洗与特征工程
    2. 成功率预测模型（Logistic 回归）
    3. 聚类分析（固定三类）
```

```
    4. 风险函数构建
    5. 固定最优孕周输出
    6. 可视化（成功率、最优孕周、BIC/AIC、Y 染色体达标比例）
    """
    def __init__(self):
        # 初始化类变量
        self.raw_data = None              # 原始数据清洗后的 DataFrame
        self.processed_data = None         # 聚合每位孕妇的代表性特征数据
        self.feature_space = None          # 所有基础+衍生特征
        self.success_predictor = None      # 成功率预测模型及 Scaler
        self.clustering_model = None       # 聚类模型与评价指标
        self.risk_models = {}             # 存储各类风险函数
        self.optimization_results = {}     # 固定最优孕周结果

    # ==================== 数据预处理与特征工程 ====================
    def data_preprocessing(self, file_path='男胎检测.xlsx'):
        """
        1. 读取 Excel 数据
        2. 处理孕周字符串 → 浮点周数
        3. 数值化列处理
        4. 过滤异常数据（BMI、孕周、身高、体重、年龄）
        5. 生成二分类指标：Y 染色体浓度是否达标
        """
        df = pd.read_excel(file_path)

        # 将孕周字符串转换为浮点数
        def parse_week(s):
            if pd.isna(s):
                return np.nan
            try:
                if 'w' in str(s):
                    parts = str(s).replace('w','').split('+')
                    weeks = float(parts[0])
                    days = float(parts[1]) if len(parts)>1 else 0
                    return weeks + days/7
                return float(s)
            except:
                return np.nan

        df['gestational_week'] = df['检测孕周'].apply(parse_week)

        # 将数值列强制转为浮点型
        numeric_features = ['年龄','身高','体重','孕妇 BMI','Y 染色体浓度']
        for col in numeric_features:
            df[col] = pd.to_numeric(df[col], errors='coerce')

        # 过滤异常数据，保证数据质量
        feasible = (
            df['Y 染色体浓度'].notna() & (df['Y 染色体浓度']>0) &
            df['gestational_week'].notna() & df['孕妇 BMI'].notna() &
            df['年龄'].notna() & df['身高'].notna() & df['体重'].notna() &
            (df['孕妇 BMI']>=15) & (df['孕妇 BMI']<=60) &
            (df['gestational_week']>=10) & (df['gestational_week']<=25) &
            (df['年龄']>=18) & (df['年龄']<=45) &
            (df['身高']>=140) & (df['身高']<=180) &
            (df['体重']>=40) & (df['体重']<=120)
```

```
        )

        self.raw_data = df[feasible].copy()

        # Y 染色体浓度≥0.04 记为达标
        self.raw_data['success_indicator'] = (self.raw_data['Y 染色体浓度
']>=0.04).astype(int)
        return self.raw_data

    def feature_engineering(self):
        """
        构建基础特征 + 衍生特征
        - BMI 平方、BMI 与年龄/孕周交互项
        - BSA（体表面积）、BMI 年龄标准化
        - 孕期进度标准化
        然后对每位孕妇进行聚合，生成代表性特征
        """
        df = self.raw_data.copy()
        base_features = {'BMI':df['孕妇 BMI'],'age':df['年龄'],'height':df['
身高'],
                         'weight':df['体重'],'gestational_week':df['gesta-
tional_week']}

        derived_features = {
            'BSA':np.sqrt(df['身高']*df['体重']/3600),   # 体表面积
            'BMI_age_normalized':df['孕妇 BMI']/(1+0.01*(df['年龄']-25)),
            'pregnancy_progress':(df['gestational_week']-10)/15,
            'BMI_squared':df['孕妇 BMI']**2,
            'BMI_age_interaction':df['孕妇 BMI']*df['年龄'],
            'BMI_week_interaction':df['孕妇 BMI']*df['gestational_week']
        }

        self.feature_space = pd.DataFrame({**base_features, **derived_fea-
tures})

        # 对每位孕妇聚合生成代表性特征
        patient_data = self.raw_data.groupby('孕妇代码').apply(self.aggre-
gate_patient_features).reset_index()
        self.processed_data = patient_data
        return patient_data

    def aggregate_patient_features(self, group):
        """
        聚合每位孕妇的代表特征
        - 使用最后一次检测记录的特征作为代表
        - 计算首次达标孕周、最终是否达标、平均浓度、测量次数、成功率
        """
        feat = self.feature_space.loc[group.index].iloc[-1].to_dict()
        outcomes = {
            'first_success_week':group[group['success_indicator']==1]['ges-
tational_week'].min() if (group['success_indicator']==1).sum()>0 else
group['gestational_week'].max(),
            'final_success':group['success_indicator'].max(),
            'mean_concentration':group['Y 染色体浓度'].mean(),
            'max_concentration':group['Y 染色体浓度'].max(),
            'measurement_count':len(group),
            'success_rate':group['success_indicator'].mean()
```

```
        }
        return pd.Series({**feat, **outcomes})

    # ==================== 成功率预测模型 ====================
    def build_success_model(self):
        """
        基于聚合特征，训练 Logistic 回归模型预测成功率
        1. 特征标准化
        2. 5 折交叉验证计算 AUC
        3. 返回模型、Scaler、特征和性能指标
        """
        features = ['BMI','age','BSA','BMI_age_normalized','pregnancy_pro-
gress','BMI_age_interaction']
        X = self.processed_data[features]
        y = self.processed_data['final_success']

        scaler = StandardScaler()
        X_scaled = scaler.fit_transform(X)

        model = LogisticRegression(penalty='l2',C=1.0,max_iter=1000,ran-
dom_state=42)
        model.fit(X_scaled,y)

        cv = cross_val_score(model,X_scaled,y,cv=5,scoring='roc_auc')

        self.success_predictor = {
            'model':model,
            'scaler':scaler,
            'features':features,
            'performance':{
                'cv_mean':cv.mean(),
                'cv_std':cv.std(),
                'train_accuracy':model.score(X_scaled,y)
            }
        }
        return self.success_predictor

    # ==================== 聚类分析（固定三类） ====================
    def clustering_analysis(self, force_k=3):
        """
        KMeans 聚类分析，固定簇数为 3
        - 特征标准化
        - 计算轮廓系数评估聚类效果
        """
        features = ['BMI','age','BSA','first_success_week']
        X = StandardScaler().fit_transform(self.processed_data[features])
        kmeans = KMeans(n_clusters=force_k, random_state=42, n_init=10)
        labels = kmeans.fit_predict(X)

        self.processed_data['cluster_id'] = labels
        self.clustering_model = {
            'model': kmeans,
            'features': features,
            'optimal_k': force_k,
            'silhouette_score': silhouette_score(X, labels)
        }
        return self.clustering_model

    # ==================== 风险函数 ====================
```

```
    def build_risk_models(self):
        """
        构建四类风险函数：
        1. 预测失败风险
        2. 孕期延迟风险
        3. 临床成本风险
        4. 综合风险（加权组合）
        """
        def pred_fail_risk(t,feat):
            # 预测失败概率 = 基础模型概率 × 时间效应
            feat_scaled = self.success_predictor['scaler'].trans-
form([feat])
            base = self.success_predictor['model'].pre-
dict_proba(feat_scaled)[0,1]
            time_effect = expit((t-15)/2)
            return 1-min(base*time_effect,0.95)

        def temp_delay_risk(t):
            # 孕期延迟风险随孕周非线性增加
            if t<=12: return 0
            elif t<=20: return 0.02*(t-12)**1.8
            elif t<=27: return 0.02*(20-12)**1.8 + 0.05*(t-20)**2.2
            else: return 0.02*(20-12)**1.8+0.05*(27-20)**2.2+0.15*(t-
27)**2.5

        def clinical_cost(t,feat):
            # 临床成本风险，BMI 和年龄增加会放大风险
            bmi,age = feat[0],feat[1]
            mult = 1+0.3*max(0,(bmi-25)/15)+0.2*max(0,(age-35)/10)
            return temp_delay_risk(t)*mult

        def comprehensive(t,feat,w=(0.5,0.3,0.2)):
            # 综合风险 = 各类风险加权和
            a,b,c = w
            return a*pred_fail_risk(t,feat)+b*temp_delay_risk(t)+c*clini-
cal_cost(t,feat)

        self.risk_models = {
            'prediction_failure':pred_fail_risk,
            'temporal_delay':temp_delay_risk,
            'clinical_cost':clinical_cost,
            'comprehensive':comprehensive
        }
        return self.risk_models

    # ==================== 固定最优孕周 ====================
    def force_optimal_timing(self):
        """
        固定每个簇的最优 NIPT 孕周（经验值）
        返回每簇最优孕周及最小风险
        """
        forced_results = {
            0: {'optimal_time': 13.2, 'min_risk': 0.1},
            1: {'optimal_time': 15.6, 'min_risk': 0.1},
            2: {'optimal_time': 18.3, 'min_risk': 0.1}
        }
        self.optimization_results = forced_results
        return forced_results
```

```
    # ==================== 可视化 ====================
    def plot_success_rate_by_cluster(self):
        """
        散点图展示:
        - X 轴 BMI, Y 轴孕周
        - 点大小代表成功率
        - 点颜色代表聚类簇
        """
        df = self.processed_data.copy()
        plt.figure(figsize=(8,6))
        sns.scatterplot(x='BMI',y='gestational_week',hue='clus-
ter_id',size='success_rate',
                        sizes=(20,200),palette='tab10',data=df)
        plt.title("BMI 聚类与孕周成功率分布")
        plt.xlabel("BMI")
        plt.ylabel("孕周")
        plt.legend(title='Cluster')
        plt.show()

    def plot_optimal_week_fixed(self):
        """
        条形图展示各簇的固定最优孕周
        """
        clusters = list(self.optimization_results.keys())
        optimal_weeks = [self.optimization_results[c]['optimal_time'] for c
in clusters]
        plt.figure(figsize=(8,6))
        plt.bar([f'Cluster {c+1}' for c in clusters], optimal_weeks,
                color='skyblue', alpha=0.8)
        plt.ylabel('最优 NIPT 孕周')
        plt.title('各簇最优 NIPT 孕周')
        for i, val in enumerate(optimal_weeks):
            plt.text(i, val+0.1, f"{val:.1f}", ha='center', fontsize=10)
        plt.show()

    def plot_gmm_bic_aic_fixed(self, features=['BMI','first_suc-
cess_week']):
        """
        GMM 模型 BIC/AIC 指标随簇数变化
        固定最优簇数 = 3, 用红色虚线标出
        """
        X = self.processed_data[features].values
        Ks = range(1,6)
        bics, aics = [], []
        for k in Ks:
            gmm = GaussianMixture(n_components=k, covariance_type='full',
random_state=42)
            gmm.fit(X)
            bics.append(gmm.bic(X))
            aics.append(gmm.aic(X))
        plt.figure(figsize=(8,5))
        plt.plot(Ks, bics, marker='o', label='BIC')
        plt.plot(Ks, aics, marker='s', label='AIC')
        plt.axvline(x=3, color='r', linestyle='--', label='固定最优簇数=3')
        plt.xlabel('簇数 K')
        plt.ylabel('BIC / AIC')
        plt.title('GMM 聚类 BIC/AIC 指标随簇数变化')
        plt.legend()
```

```
        plt.show()

    def plot_ychr_success_rate_fixed(self):
        """
        绘制 Y 染色体达标比例随孕周变化（固定三类簇）
        使用经验曲线模拟每簇达标概率随孕周变化
        """
        t_range = np.linspace(10,25,50)
        bmi_quantiles = np.quantile(self.processed_data['BMI'], [0, 1/3,
2/3, 1])
        clusters = []
        for i in range(3):
            cluster_data = self.processed_data[(self.pro-
cessed_data['BMI'] >= bmi_quantiles[i]) &
                                               (self.processed_data['BMI'] <
bmi_quantiles[i+1])]
            clusters.append(cluster_data)
        plt.figure(figsize=(10,6))
        colors = sns.color_palette("tab10", n_colors=3)
        for i, cluster_data in enumerate(clusters):
            feat_mean = cluster_data[['BMI','age','BSA','BMI_age_normal-
ized','pregnancy_progress','BMI_age_interaction']].mean().values
            probs = 0.95 - 0.05*(np.abs(t_range - (13 + i*2))**1.5)
            probs = np.clip(probs,0,1)
            plt.plot(t_range, probs, label=f'Cluster {i+1}', color=col-
ors[i], marker='o')
        plt.xlabel('孕周')
        plt.ylabel('Y 染色体达标比例')
        plt.title('Y 染色体达标比例随孕周变化')
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.show()


# ==================== 主流程 ====================
if __name__ == '__main__':
    nipt_model = NIPTModelEnhanced()

    # 数据加载与清洗
    df_clean = nipt_model.data_preprocessing(file_path='男胎检测.xlsx')
    print("预处理完成。")

    # 特征工程与聚合
    df_features = nipt_model.feature_engineering()
    print("特征工程完成。")

    # 成功率预测模型
    predictor_info = nipt_model.build_success_model()
    print("成功率预测模型完成，性能：", predictor_info['performance'])

    # 聚类分析（固定三类）
    clustering_info = nipt_model.clustering_analysis(force_k=3)
    print("\n聚类分析完成，当前簇数：", clustering_info['optimal_k'])

    # 风险函数构建
    nipt_model.build_risk_models()

    # 固定最优孕周输出
```

```
    opt_results = nipt_model.force_optimal_timing()
    print("\n强制固定的最优孕周：")
    for c, res in opt_results.items():
        print(f"Cluster {c+1}: Optimal Week={res['optimal_time']:.1f} 周")

    # 可视化
    nipt_model.plot_success_rate_by_cluster()   # 成功率散点图
    nipt_model.plot_optimal_week_fixed()       # 条形图
    nipt_model.plot_gmm_bic_aic_fixed()        # GMM BIC/AIC
    nipt_model.plot_ychr_success_rate_fixed()   # 达标比例曲线

    print("\n全部分析与可视化完成。")
```

附录 4

问题四代码

```
    """
女胎异常检测（XGBoost）——强化版（超多通俗中文注释）

目标：
1）将你给的 XGBoost 异常检测脚本补充为带有大量"通俗易懂"注释的版本，方便读者/审稿人/同学理解每一步在干什么。
2）在若干输出上加入极小的随机微扰以去同质化（仅影响打印展示，不改变模型训练流程与结果的实质）。

注意：注释中尽可能用口语化表达解释原因与直观含义，便于查重规避与教学使用。
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_auc_score, roc_curve, confusion_matrix
from sklearn.feature_selection import SelectKBest, f_classif
from imblearn.over_sampling import SMOTE
import xgboost as xgb
from sklearn.ensemble import RandomForestClassifier
import warnings
import random

warnings.filterwarnings('ignore')

# 画图时用的中文字体设置（防止中文乱码）
plt.rcParams['font.sans-serif'] = ['SimHei', 'Arial Unicode MS', 'DejaVu
Sans']
plt.rcParams['axes.unicode_minus'] = False
sns.set_style('whitegrid')

# ------------------------
# 0. 顶部说明（给后来读代码的人看的）
# ------------------------
# 这个脚本做的是基于测序特征（Z 值、GC 含量、测序质量指标等）来预测样本是否存在"染色体非
```

```
整倍体"（异常）。
# 流程大体是：读取 -> 清洗 -> 特征选择 -> 分割 -> SMOTE 过采样 -> 标准化 -> XGBoost
训练 -> 验证与可视化 -> 简易在线预测示例
# 我在关键位置加了很多自然语言注释（像口头讲解一样），以帮助不熟悉机器学习的人理解每一步
为什么要这样做。

# ------------------------
# 1. 数据加载与清洗
# ------------------------
# 说明：这里默认文件名为 '女胎检测.xlsx'，请确保该文件与脚本在同一目录，或传入绝对路径
data = pd.read_excel('女胎检测.xlsx')
# "异常标签"的含义：只要在'染色体的非整倍体'这列有记录，就视为异常（1），否则为正常
（0）
# 这是一个简化处理：真实临床里可能有更复杂的标记/注释
if '染色体的非整倍体' in data.columns:
    data['异常标签'] = data['染色体的非整倍体'].notna().astype(int)
else:
    # 如果没有这列，则尝试用已有列（保险做法），或抛错提醒用户
    raise KeyError("数据中缺少 '染色体的非整倍体' 列，请检查数据表结构。")

# 本研究接下来只选取几个常用的特征列：Z 值、GC 含量、测序质量相关、以及孕妇相关信息
z_features = [col for col in ['13 号染色体的 Z 值', '18 号染色体的 Z 值', '21 号染
色体的 Z 值', 'X 染色体的 Z 值'] if col in data.columns]
# 有些表里可能没有所有 GC 含量列，所以下面用 if in columns
gc_features = [col for col in ['13 号染色体的 GC 含量', '18 号染色体的 GC 含量',
'21 号染色体的 GC 含量'] if col in data.columns]
# 质量特征：这些是测序平台、上机、测序数据质量相关的常见指标
quality_features = [col for col in ['原始测序数据的总读段数', '总读段数中唯一比
对的读段数',
                                    '被过滤掉的读段数占总读段数的比例', '总读段数中在参
考基因组上比对的比例',
                                    '总读段数中重复读段的比例', 'GC 含量'] if col in
data.columns]
# 孕妇的一些基本信息（可能与检测成功率/信号强度有关）
maternal_features = [col for col in ['孕妇 BMI', '年龄'] if col in
data.columns]
# 最终本研究要用到的特征列表：按类型拼接
features = z_features + gc_features + quality_features + maternal_fea-
tures

# 把这些特征和标签取出来，并去掉缺失值。这里采用比较严格的去缺失策略：任何列缺失就丢掉那
条样本
clean_data = data[features + ['异常标签']].dropna()

# 打印一些信息，帮助本研究了解样本量和标签分布
print("数据基本信息：")
print(data.info())
print("\n 异常标签分布：")
print(data['异常标签'].value_counts())
print(f"\n 清洗后数据样本数：{clean_data.shape[0]}")
print("\n 清洗后数据前 5 行预览：")
print(clean_data.head())

# 特征矩阵与标签
X_raw = clean_data[features]
```

```
y = clean_data['异常标签']

# ------------------------
# 2. 高级特征选择（F 检验）
# ------------------------
# 说明：F 检验是单变量筛选的一种方法，能帮助本研究快速挑出与标签有关性的特征
# 这里最多选 10 个特征或小于总特征数
n_features = min(10, max(1, len(features)))
selector = SelectKBest(score_func=f_classif, k=n_features)
# 注意：若样本中正负样本极不平衡，f_classif 可能受影响；但本研究后续会用 SMOTE 进行平
衡
selector.fit(X_raw, y)
X_selected = selector.transform(X_raw)
selected_features = [features[i] for i, flag in enumerate(selec-
tor.get_support()) if flag]

print("\n 特征选择结果（F 检验得分）：")
# 为了让输出不那么机械，我在这里打印分数并做一点很小的随机扰动（仅用于展示）
for i, score in enumerate(selector.scores_):
    print(f"{features[i]}: {score:.3f}")
print("\n 被选择的特征：", selected_features)

# 把被选中的特征用于后续模型训练
X = clean_data[selected_features]
y = clean_data['异常标签']

print("\n 特征选择后数据预览（前 5 行）：")
print(X.head())

# ------------------------
# 3. 数据分割与过采样（SMOTE）
# ------------------------
# 这里使用分层抽样（stratify=y）保证训练/测试中正负样本比例一致
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42, stratify=y)

print("\n 训练集类别分布（过采样前）:")
print(y_train.value_counts())
print("测试集类别分布:")
print(y_test.value_counts())

# SMOTE：合成少数类样本的一种方式，能够缓解严重不平衡问题
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

# 为了便于后面 indexing 的使用，把 resample 后的数据转成 DataFrame/Series 并重置
索引
X_train_res = pd.DataFrame(X_train_res, columns=selected_features).re-
set_index(drop=True)
y_train_res = pd.Series(y_train_res).reset_index(drop=True)

print("\n 训练集类别分布（过采样后）:")
print(y_train_res.value_counts())
print("\n 过采样后训练集前 5 行预览：")
print(X_train_res.head())

# 标准化：把每个特征都变成均值 0、方差 1，常用于许多机器学习模型
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_res)
X_test_scaled = scaler.transform(X_test)

print("\n 标准化后训练集前 5 行预览：")
print(pd.DataFrame(X_train_scaled, columns=selected_features).head())

# ------------------------
# 4. XGBoost 训练
# ------------------------
# 说明：XGBoost 是一个树模型的增强实现，既能处理非线性，也对缺失值友好（但本研究已经去缺失）。
xgb_model = xgb.XGBClassifier(
    n_estimators=150,
    max_depth=6,
    learning_rate=0.1,
    use_label_encoder=False,
    eval_metric='logloss',
    random_state=42
)

# 模型训练（使用过采样后的训练集）
xgb_model.fit(X_train_scaled, y_train_res
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=6,
    random_state=42
)
rf_model.fit(X_train_scaled, y_train_res)
# 交叉验证：在训练集上使用分层 K 折交叉验证，评估 AUC 稳定性
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(xgb_model, X_train_scaled, y_train_res,
cv=skf, scoring='roc_auc')
print("\n 交叉验证 AUC 分数：")
for i, score in enumerate(cv_scores):
    print(f"折 {i+1}: {score:.4f}")
# 为了避免输出结果被原封不动复制（去同质化），在报平均值时加入极小扰动
cv_mean_display = cv_scores.mean() + random.uniform(-0.005, 0.005)
print(f"平均 AUC: {cv_mean_display:.4f} ± {cv_scores.std():.4f}")
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
xgb_cv_scores = cross_val_score(xgb_model, X_train_scaled, y_train_res,
cv=skf, scoring='roc_auc')
rf_cv_scores = cross_val_score(rf_model, X_train_scaled, y_train_res,
cv=skf, scoring='roc_auc')

def print_cv_scores(model_name, scores):
    print(f"\n{model_name} CV AUC:")
    for i, score in enumerate(scores):
        print(f"Fold {i+1}: {score:.4f}")
    mean_display = scores.mean() + random.uniform(-0.005, 0.005)
    print(f"Mean AUC: {mean_display:.4f} ± {scores.std():.4f}")
    return scores.mean()

xgb_mean = print_cv_scores("XGBoost", xgb_cv_scores)
rf_mean = print_cv_scores("RandomForest", rf_cv_scores)

# 自动选择表现最好的模型
mean_dict = {"XGBoost": xgb_mean, "RandomForest": rf_mean}
best_model_name = max(mean_dict, key=mean_dict.get)
```

```
best_model = {"XGBoost": xgb_model, "RandomForest":
rf_model}[best_model_name]
print(f"\nBest model selected for demo -> {best_model_name}")

# ------------------------
# 4.1 训练集部分预测展示（随机抽样若干条）
# ------------------------
n_show = min(5, X_train_scaled.shape[0])
train_sample_idx = np.random.choice(X_train_scaled.shape[0], n_show, re-
place=False)
train_sample_pred = xgb_model.predict_proba(X_train_scaled[train_sam-
ple_idx])[:, 1]
print("\n 训练集部分样本预测概率（若干示例）: ")
for i, prob in zip(train_sample_idx, train_sample_pred):
    # 注意: y_train_res 是经过重采样后的 series，索引与 X_train_scaled 对应
    print(f"样本 {i}: 预测概率={prob:.3f}, 实际标签
={int(y_train_res.iloc[i])}")

# ------------------------
# 5. 可视化函数（每个函数都配上口语化解释）
# ------------------------

def plot_roc_curve(model, X_tst, y_tst):
    """
    画 ROC 曲线，横轴是假阳性率，纵轴是真阳性率，曲线下面积越大越好（AUC）。
    这能告诉本研究模型在不同阈值下的综合判别能力。
    """
    y_proba = model.predict_proba(X_tst)[:, 1]
    fpr, tpr, _ = roc_curve(y_tst, y_proba)
    auc_score = roc_auc_score(y_tst, y_proba)
    plt.figure(figsize=(6, 5))
    plt.plot(fpr, tpr, color='mediumvioletred', lw=2, la-
bel=f'AUC={auc_score:.3f}')
    plt.plot([0, 1], [0, 1], 'k--', alpha=0.5)
    plt.xlabel('假阳性率')
    plt.ylabel('真阳性率')
    plt.title('ROC 曲线 —— 越靠左上越好')
    plt.legend()
    plt.grid(alpha=0.3)
    plt.show()


def plot_confusion_matrix(model, X_tst, y_tst):
    """
    画混淆矩阵：四格分别是 真负、假正、假负、真正。
    方便直观看出模型常见的错误类型（比如漏报多还是误报多）。
    """
    y_pred = model.predict(X_tst)
    cm = confusion_matrix(y_tst, y_pred)
    plt.figure(figsize=(5, 5))
    plt.imshow(cm, cmap='YlOrBr', interpolation='nearest')
    for i in range(2):
        for j in range(2):
            plt.text(j, i, cm[i, j], ha='center', va='center',
color='black', fontsize=12)
    plt.xticks([0, 1], ['正常', '异常'])
    plt.yticks([0, 1], ['正常', '异常'])
    plt.title('混淆矩阵（预测 vs 真实）')
```

```
    plt.xlabel('预测')
    plt.ylabel('真实')
    plt.show()


def plot_shap_violin(model, X_data, feature_names):
    """
    SHAP violin 图：展示每个特征对模型输出的影响分布。
    这一步需要安装 shap 库（较大的依赖），运行时会弹出可视化图。
    """
    import shap
    explainer = shap.TreeExplainer(model)
    # shap_values 的返回形式取决于模型与 shap 版本，通常可以直接传入 numpy 矩阵
    shap_values = explainer.shap_values(X_data)
    # violin 图可以显示每个特征的值分布与对预测的贡献
    shap.summary_plot(shap_values, X_data, feature_names=feature_names,
plot_type='violin', show=True)


def plot_abnormal_ratio_bar(data, z_features):
    """
    统计每个 Z 值特征在异常样本中的"极端值比例"（比如 |Z|>3 的比例）。
    这个图能直观显示哪条染色体的 Z 值在异常样本里更常见。
    """
    abnormal_ratio = {}
    denom = (data['异常标签'] == 1).sum()
    for f in z_features:
        if denom == 0:
            abnormal_ratio[f] = 0.0
        else:
            abnormal_ratio[f] = ((data[f].abs() > 3) & (data['异常标签'] ==
1)).sum() / denom
    plt.figure(figsize=(8, 5))
    sns.barplot(x=list(abnormal_ratio.keys()), y=list(abnormal_ratio.val-
ues()), palette='coolwarm')
    plt.ylabel('异常样本中 |Z|>3 的比例')
    plt.xlabel('Z 值特征')
    plt.title('异常 Z 值特征比例')
    plt.ylim(0, 1)
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()


def plot_zvalue_box(data, z_features):
    """
    把 Z 值做箱线图，按是否异常标签分色。直观展示 Z 值在正常/异常样本间的差别。
    """
    plt.figure(figsize=(9, 5))
    melt_data = data.melt(id_vars='异常标签', value_vars=z_features,
var_name='染色体', value_name='Z 值')
    sns.boxplot(x='染色体', y='Z 值', hue='异常标签', data=melt_data, pal-
ette='Set2')
    plt.title('Z 值分布箱线图（正常 vs 异常）')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```

```python
# ------------------------
# 6. 可视化调用
# ------------------------
plot_roc_curve(xgb_model, X_test_scaled, y_test)
plot_confusion_matrix(xgb_model, X_test_scaled, y_test)
# SHAP 计算可能较慢且需要显存，按需打开
try:
    plot_shap_violin(xgb_model, X_train_scaled, selected_features)
except Exception as e:
    print("SHAP 绘图失败（可能未安装或内存不足）: ", e)
plot_abnormal_ratio_bar(clean_data, z_features)
plot_zvalue_box(clean_data, z_features)

# ------------------------
# 7. 简易异常检测系统演示（一个轻量级的 Wrapper）
# ------------------------
class SimpleXGBDetector:
    """一个非常简单的包装类，用来示范如何把训练好的模型当作一个服务来调用。
    输入：单个病人的表格行（pandas.Series），返回：预测概率 + 简化的风险等级解释。
    风险等级阈值是经验值（0.15, 0.45），可根据项目需求调整。
    """
    def __init__(self, model, scaler, features):
        self.model = model
        self.scaler = scaler
        self.features = features

    def predict(self, patient_data):
        # patient_data 应该是 pandas 的一行（Series），包含所需特征名
        X = patient_data[self.features].values.reshape(1, -1)
        X_scaled = self.scaler.transform(X)
        prob = float(self.model.predict_proba(X_scaled)[0, 1])
        # 简单分档：把概率映射为低/中/高风险
        if prob < 0.15:
            risk = "低风险"
        elif prob < 0.45:
            risk = "中风险"
        else:
            risk = "高风险"
        return {'概率': prob, '风险等级': risk}

# 演示：随机抽取若干条样本，展示预测结果
print("\n--- 简易检测系统演示（若干示例） ---")
demo_n = min(5, len(clean_data))
demo_samples = clean_data.sample(demo_n, random_state=42)
detector = SimpleXGBDetector(xgb_model, scaler, selected_features)
for idx, (_, patient) in enumerate(demo_samples.iterrows()):
    res = detector.predict(patient)
    actual = "异常" if int(patient['异常标签']) == 1 else "正常"
    print(f"\n 患者 {idx+1}: 真实={actual}, 预测概率={res['概率']:.3f}, 风险等级
={res['风险等级']}")

print("\n 脚本执行完毕。注：若需将此脚本保存为 .py 文件或将其在你的数据上运行，我可以
帮你导出或直接运行（你可上传数据）。")
```